

Process Research & Architecture Specification

پژوهش و طراحی معماری — Process Builder / Workflow Engine — پروژه org_platform

Generated: 2026-08-19 · Source: docs/process-research/

۱. ۰۱ – خلاصه‌ی مدیریتی (Executive Summary)
۲. ۰۲ – مفاهیم پایه‌ی (Process (Concept Glossary)
۳. ۰۳ – Industry Taxonomy
۴. ۰۴ – Cross-Functional و Domain Taxonomy طراحی
۵. ۰۵ – Process Taxonomy (فرایندهای واقعی هر حوزه)
۶. ۰۶ – Process Patterns (الگوهای تکرارشونده)
۷. ۰۷ – کاتالوگ Node های Workflow Builder
۸. ۰۸ – معماری پیشنهادی Process Engine
۹. ۰۹ – Data Model پیشنهادی
۱۰. ۱۰ – Process Library
۱۱. ۱۱ – ارتباط Process Builder با سایر ماژول‌های محصول
۱۲. ۱۲ – مدل SLA و Escalation
۱۳. ۱۳ – مدل Role و Assignment
۱۴. ۱۴ – Rule Engine (جدا از Workflow)
۱۵. ۱۵ – مدل Versioning (Template در برابر Instance)
۱۶. ۱۶ – Process Completeness Analysis و بازبینی نهایی کیفیت
۱۷. ۱۷ – توصیه‌های محصول: MVP در برابر V2 در برابر Enterprise، و مقایسه با پروژه‌ی فعلی

۱۰ – خلاصه‌ی مدیریتی (Executive Summary)

این سند ورودی کل Research است اما آخرین سندی است که نوشته شد – چون فقط بعد از تکمیل بقیه می‌شد جمع‌بندی درست کرد. اگر برای اولین بار این پوشه را می‌خوانید، از همین‌جا شروع کنید.

این Research چه چیزی تولید کرد

بر مبنای تحقیق چندمنبعی روی APQC PCF (با استخراج مستقیم از PDF رسمی، نه خلاصه‌ی ثانویه)، استاندارد BPMN/DMN، و بررسی معماری واقعی پنج پلتفرم صنعتی (Camunda، ServiceNow، Salesforce، Pega، Zendesk/AWS Step Functions/n8n)، این پوشه یک **Data + کامل Taxonomy** **Model + Process Library** با ۵۱ فرایند واقعی (۲۰ نمونه‌ی کاملاً باز شده) تولید کرد که مستقیماً قابل تبدیل به Migration، Seed Data، و طراحی Builder است – بدون آنکه تیم توسعه مجبور شود دوباره از صفر درباره‌ی ماهیت Process‌ها تحقیق کند.

هیچ کدی در پروژه تغییر نکرد. فقط این پوشه (docs/process-research/) و data/process- library/ ساخته شدند.

پاسخ به ۱۰ سؤال کلیدی

۱. یک **Process Builder** واقعاً **Generic** چه ویژگی‌هایی باید داشته باشد؟ باید گراف Node/Edge باشد (نه Code-محور مثل Temporal) چون کاربر نهایی توسعه‌دهنده نیست؛ Approval باید Node مستقل با پشتیبانی Multi-level/Parallel باشد (نه یک Task ساده)؛ Rule Engine باید کاملاً از گراف جدا باشد (الگوی DMN)؛ و باید از ابتدا Template/Version/Instance را جدا نگه دارد (نگاه کنید [ه‌ا](#)).

۲. چه چیزهایی باید در **Core Engine** باشند؟ دقیقاً ۱۵ عنصر «Universal Process Core» شناسایی شده در [ه‌ا](#): Request، Form، Task، Assignment، Approval، Decision، Notification، SLA، Escalation، Document، Comment، Attachment، Audit Log، Status، History.

۳. چه چیزهایی نباید **Hard-code** شوند؟ هر چیزی که در data/process-library/*.json به صورت Seed آمده – Domain، Process Type، Node Catalog، Assignment Method، Hit Policy

– همه باید جدول Database باشند، نه Enum ثابت در کد Backend (نگاه کنید [data-model-۰۹](#)).

۴. Process Library چگونه باید طراحی شود؟ روی اسکلت ۱۳ Category تأییدشده‌ی APQC PCF، با یک لایه‌ی Process Type ثانویه (بر اساس رفتار، نه Domain) – نگاه کنید [process-library-۱۰](#).

۵. چگونه می‌توان صدها/هزاران Process را بدون پیچیده‌شدن محصول مدیریت کرد؟ با تفکیک صریح Taxonomy (طبقه‌بندی، ثابت و کم‌تغییر) از Workflow (اجرا، هرچقدر لازم است متنوع) – یک Process Taxonomy می‌تواند صدها Workflow Template مختلف در سازمان‌های مختلف داشته باشد بدون آنکه خود Taxonomy رشد کند (نگاه کنید تفاوت Process/Workflow در [۰۲](#)).

۶. چه چیزهایی باید Template باشند؟ `workflow_templates` + `workflow_versions` (گراف)، `business_rule_versions`، `form_versions` – همه‌ی این‌ها Snapshot غیرقابل‌تغییر بعد از انتشارند (نگاه کنید [versioning-۱۵](#)).

۷. چه چیزهایی Runtime باشند؟ `work_items`، `workflow_instances`، `process_variables` – همیشه در حال تغییر تا Completion.

۸. چه چیزهایی باید Configurable باشند؟ Assignment Rule، SLA Target، Escalation Level، Business Rule Threshold – همه از طریق Data، نه Deploy مجدد کد، قابل‌تغییر (نگاه کنید [۱۲](#)، [۱۳](#)، [۱۴](#)).

۹. چه چیزهایی باید Plugin/Integration باشند؟ Webhook/API Call، اتصال به Calendar/Project/Task Manager/Notification (نگاه کنید [integration-model-۱۱](#)) – این‌ها مرزهای محصول با دنیای بیرون/سایر ماژول‌ها هستند، باید Contract صریح داشته باشند نه اتصال مستقیم به جدول داخلی ماژول دیگر.

۱۰. MVP چه بخش‌هایی را باید داشته باشد؟ Template/Version پایه + Node‌های Audit و Form Builder + Start/End/Task/Approval/Decision/Branch/Parallel/Merge اتصال به Log موجود پروژه – همه چیزهایی که روی زیرساخت فعلی `org_platform` بدون ریسک امنیتی جدید قابل‌ساخت است (نگاه کنید جدول کامل در [۱۷](#)).

یافته‌های کلیدی که مسیر تصمیم را عوض کردند

۱. APQC PCF واقعاً یک پایه‌ی اثبات‌شده است، نه یک انتخاب پرریسک – SAP Signavio، MEGA، Interfacing، و حتی یک ابزار Workflow Automation (Workato) همین الگو را دنبال کرده‌اند (نگاه

کنید [۵۳](#)). این ریسک «کپی کردن بدون تحلیل» را که کاربر صریحاً نگرانیش بود، رفع می‌کند – چون خود این Research نشان داد APQC فقط اسکلت طبقه‌بندی می‌دهد و مرز اجرا (Workflow) را عمده‌اً خالی می‌گذارد؛ آن بخش کاملاً کار این محصول است.

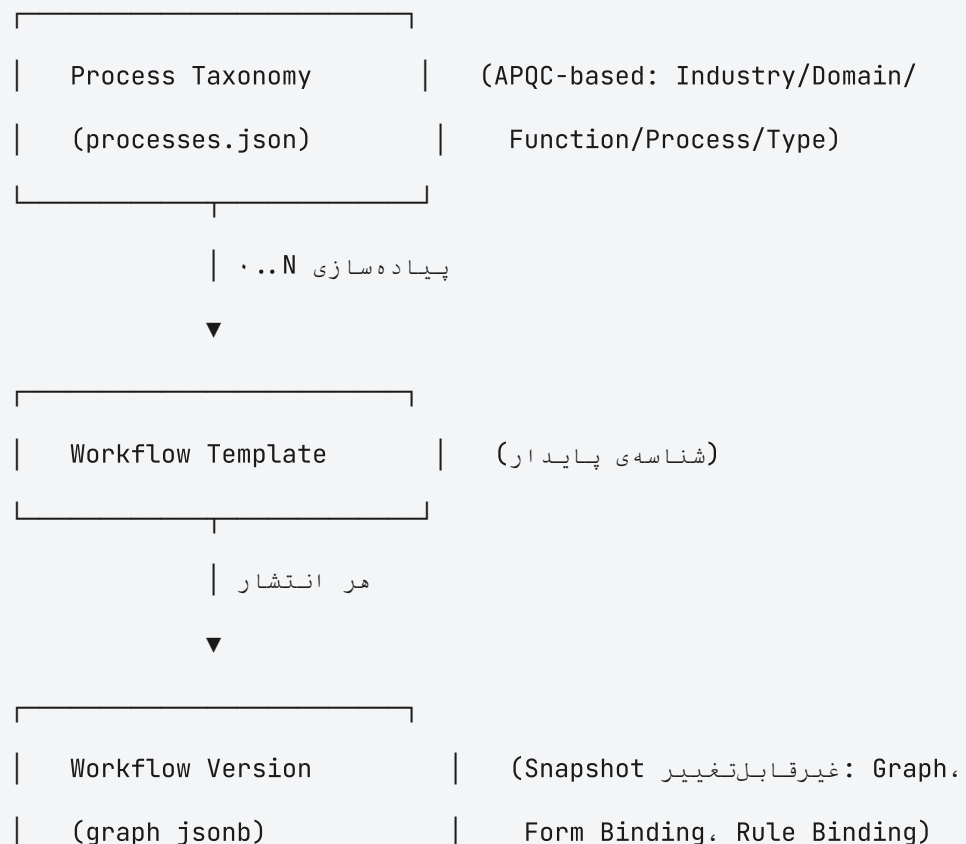
2. «Definition» و «Version» باید یک خانوادگی رکورد باشند، نه دو لایه‌ی جدا – این برخلاف پیشنهاد اولیه‌ی خود کاربر بود اما با شواهد مستقیم از چهار موتور واقعی (Camunda 7/8، Step Functions، Temporal) اصلاح شد (نگاه کنید [۱۵](#)).

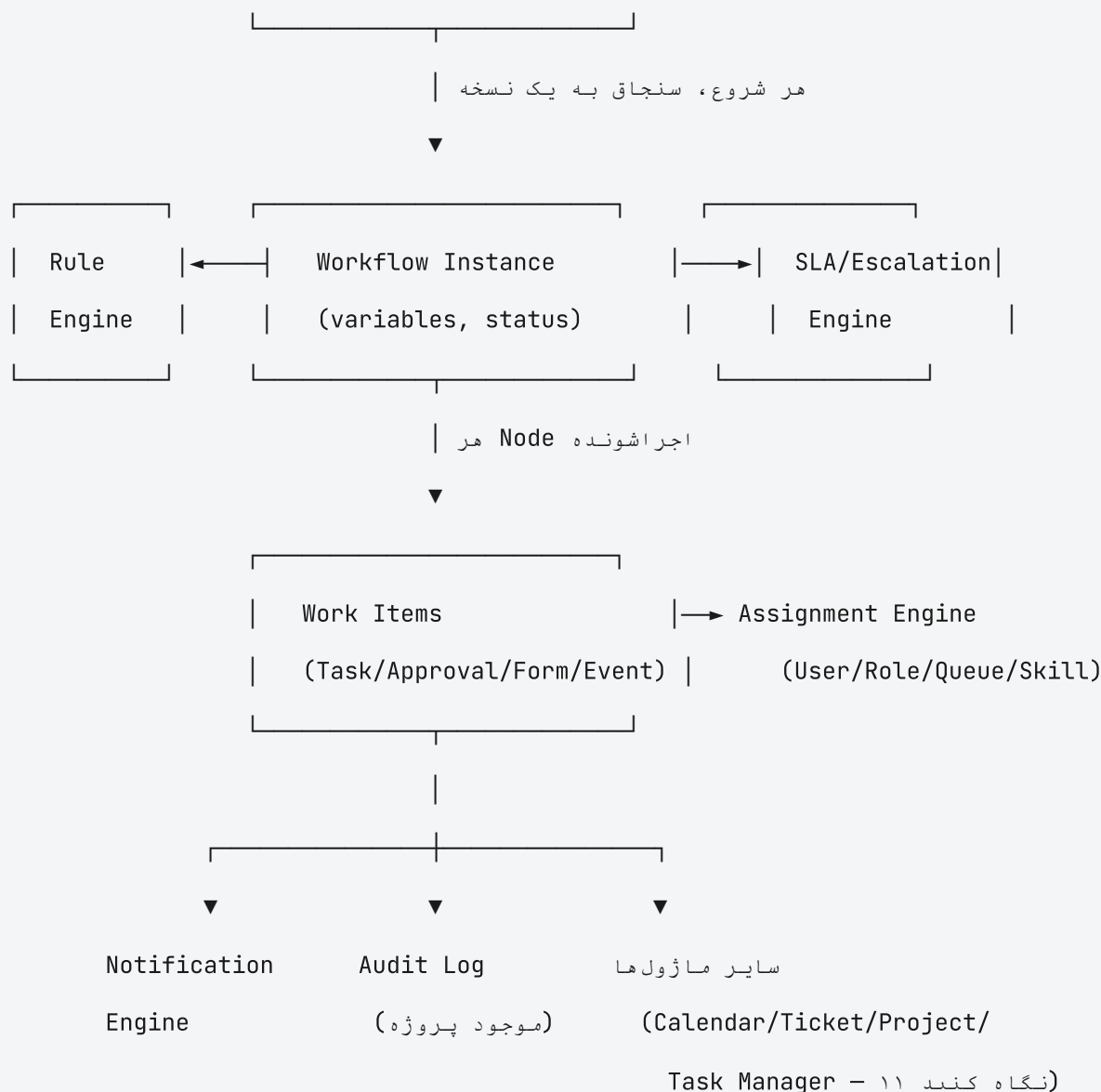
3. `workflow_nodes / workflow_edges` نباید جدول SQL نرمال‌شده باشند – هیچ موتور واقعی بررسی‌شده این کار را نمی‌کند؛ گراف باید JSON باشد (نگاه کنید [۵۹](#)).

4. Escalation سه معماری رقیب معتبر دارد، نه یک استاندارد – این محصول باید آگاهانه ترکیب دو مورد را انتخاب کند (نگاه کنید [۱۲](#))، نه اینکه فرض کند یک «راه درست» وجود دارد.

5. پروژه‌ی فعلی `org_platform` از قبل زیرساخت غیرمنتظره‌ای برای این هسته دارد – `manager_id`، `audit_log` Append-only، Form Builder Foundation واقعی MVP خیلی کمتر از صفر است (نگاه کنید [۱۷](#) بخش پ).

دیاگرام معماری نهایی پیشنهادی





توصیه‌ی نهایی

شروع با MVP جدول [product-recommendations-IV](#) بخش الف – روی زیرساخت `org_members / form_templates / audit_log` موجود پروژه، بدون لمس کد فعلی. اولین قدم واقعی پیاده‌سازی (بعد از تأیید این Research توسط شما) باید `workflow_templates / workflow_versions / workflow_instances / work_items` طبق [data-model-۰۹](#) و [versioning-۱۵](#) به‌عنوان Migration باشد – چون هر تصمیم دیگری (Node Catalog، Assignment، SLA) روی همین سه/چهار جدول سوار می‌شود.

[Concepts](#) · [۰۳-Industry Taxonomy](#) · [۰۴-Domain Taxonomy](#) · [۰۵--۰۲ فهرست کامل اسناد:](#)
[Process Taxonomy](#) · [۰۶-Patterns](#) · [۰۷-Node Catalog](#) · [۰۸-Engine Architecture](#) · [۰۹-Data Model](#) · [۱۰-Process Library](#) · [۱۱-Integration Model](#) · [۱۲-SLA & Escalation](#) · [۱۳-Role & Assignment](#) · [۱۴-Rule Engine](#) · [۱۵-Versioning](#) · [۱۶-Gap Analysis](#) · [۱۷-Product Recommendations](#)

داده‌ی ۱۳ – `data/process-library/` **Seed:** فایل -process- JSON (industries, domains, process-types, processes, process-steps, workflow-patterns, workflow-nodes, roles, forms, documents, kpis, slas, sample-full-processes)

۰۲ – مفاهیم پایه‌ی Process (Concept Glossary)

این سند مبنای واژگانی کل Research است. هر سند دیگر در این پوشه این تعاریف را به عنوان مرجع فرض می‌کند. زبان مرجع (Canonical) برای شناسه‌ها انگلیسی است؛ توضیح فارسی برای فهم کاربردی آمده.

هر واژه سه بخش دارد: **تعریف**، **چرا این تعریف** (مبتنی بر کدام منبع/استدلال)، و **اشتباه رایج** (چیزی که معمولاً با آن قاطی می‌شود).

۱. سلسله‌مراتب سازمانی (چهارچوب، نه داده)

اصطلاح	تعریف
Organization	یک Tenant مستقل در SaaS – واحدی که داده‌اش کاملاً از سایر Tenant ها ایزوله است. معادل organizations در پروژه‌ی فعلی.
Industry	صنعتی که یک Organization در آن فعالیت می‌کند (مثلاً Healthcare، Retail). یک برچسب طبقه‌بندی روی Organization است، نه یک سطح در سلسله‌مراتب فرایند.
Business Domain	حوزه‌ی کاری Cross-Functional که در همه‌ی صنایع تکرار می‌شود (HR، Finance، Procurement، IT، ...). معادل تقریبی APQC Category (سطح ۱).
Department	واحد سازمانی واقعی داخل یک Organization خاص (مثلاً «واحد منابع انسانی شرکت X»). برخلاف Domain، Department به یک Organization خاص متعلق است، نه یک مفهوم عمومی.
Function	زیرمجموعه‌ی یک Domain – گروهی از فرایندهای هم‌راستا (مثلاً درون «Finance» Domain Function های «Accounts Payable»، «Accounts Receivable»). معادل تقریبی APQC Process Group (سطح ۲).
Business Capability	توانایی‌ای که سازمان دارد صرف‌نظر از اینکه چگونه اجرا می‌شود (مثلاً «توانایی مدیریت درخواست مرخصی»). Capability با Process فرق دارد: Capability یعنی «می‌تواند»، Process یعنی «چگونه واقعاً انجام می‌دهد». یک Capability می‌تواند صفر، یک یا چند Process پیاده‌سازی شده داشته باشد.

اصطلاح	تعریف
Value Stream	زنجیره‌ی End-to-End از فعالیتهایی که مستقیماً برای یک ذی‌نفع (معمولاً مشتری) ارزش تولید می‌کنند؛ معمولاً چند Domain را قطع می‌کند (مثلاً «از سفارش تا تحویل» چند واحد را درگیر می‌کند). Value Stream عمودی به مشتری نگاه می‌کند؛ Domain افقی به تخصص سازمانی.

۲. سلسله‌مراتب طبقه‌بندی فرایند (Taxonomy، بر پایه‌ی APQC PCF – نگاه کنید به ۵۳)

اصطلاح	تعریف	معادل APQC PCF (تأییدشده از منبع اصلی)
Process Category	بالاترین سطح طبقه‌بندی (مثلاً «Manage Financial Resources»).	Level 1 – Category (کد یک‌رقمی، مثل ۹.۰)
Process Group	گروهی از فرایندهای هم‌جهت داخل یک Category.	Level 2 – Process Group (کد دو-رقمی، مثل ۹.۱)
Business Process	یک فرایند مشخص با شروع/پایان قابل‌تشخیص که یک نتیجه‌ی کسب‌وکاری تولید می‌کند (مثلاً «Process Accounts Payable»).	Level 3 – Process (مثل ۹.۱.۱)
Sub Process	تجزیه‌ی اختیاری یک Process به Process‌های کوچک‌تر با همان ماهیت (خود APQC هم گاهی این کار را می‌کند، مثلاً ۷.۳.۴ زیر ۷.۳).	تعریف رسمی APQC ندارد؛ یک لایه‌ی اختیاری میانی است که خود محصول اضافه می‌کند.
Activity	یک گام کلیدی برای اجرای یک Process.	Level 4 – Activity (مثل ۹.۱.۱.۱)
Task	ریزترین واحد کار برای اجرای یک Activity؛ طبق تعریف صریح APQC «به‌طور یک‌دست Level-بندی نشده» و بین سازمان‌ها/صنایع متفاوت است.	Level 5 – Task

نکته‌ی کلیدی از Research (تأییدشده از متن اصلی APQC): PCF صراحتاً ترتیب اجرا را کدگذاری نمی‌کند («numbered ascendingly... does not imply order»). یعنی Taxonomy فقط **طبقه‌بندی** است، نه **نقشه‌ی اجرا**. مرز دقیق بین Taxonomy (این سند + ۰۳/۰۴/۰۵) و Execution (سند ۰۷ و ۰۸) همین‌جاست: از Task به پایین، مسئولیت محصول شروع می‌شود، نه APQC.

۳. واحدهای اجرا (Runtime) – چیزی که واقعاً اتفاق می‌افتد

اصطلاح	تعریف	چرا از مفهوم بالا جداست
Workflow	نمایش اجرایی/قابل ماشین خواندن یک Business Process – گراف واقعی از Node/ها/Edge ها که موتور اجرا می کند.	Process «چه کاری باید انجام شود و چرا» را طبقه بندی می کند؛ Workflow «دقیقاً با چه ترتیب/شرط/Actor» آن را اجرا می کند. یک Process می تواند چند پیاده سازی Workflow متفاوت در سازمان های مختلف داشته باشد (نگاه کنید به تفاوت Process/Workflow در بخش ۴).
Case	یک نمونه ای در حال پیگیری از یک فرایند غیرخطی/باز-تا-بسته که معمولاً چند Work Item مستقل درون خودش دارد و پایانش از قبل مشخص نیست (مثلاً یک پرونده ای بازرسی که ممکن است چند بار باز/بسته شود).	برخلاف Workflow Instance که مسیر از پیش تعریف شده دارد، Case بیشتر شبیه یک «پوشه» است که فعالیت های نامنظم به آن اضافه می شوند (الگوی Pega Case Management).
Project	مجموعه ای از Task/ها/Milestone های یک بار مصرف با هدف، بودجه و تاریخ پایان مشخص که برای رسیدن به یک خروجی منحصر به فرد اجرا می شود.	Process تکرارپذیر است (صدها بار اجرا می شود با همان الگو)؛ Project یک بار برای یک هدف خاص اجرا می شود، حتی اگر از یک Template پروژه استفاده کند.
Workflow Instance	یک اجرای مشخص و زمان دار از یک Workflow Definition (نسخه ای مشخص) – معادل یک ردیف در workflow_instances.	نگاه کنید به versioning-۱۵ برای تفاوت دقیق با Workflow Version.
Work Item	واحد کار قابل تخصیص به یک Actor درون یک Instance – می تواند Task، Approval، Form یا Event باشد.	Work Item چتر بالاتر از Task/Approval/Form است؛ هر Node اجرایی که به یک Actor نیاز دارد یک Work Item تولید می کند.
Request	ورودی اولیه ای که یک Instance را آغاز می کند – بیان نیاز توسط یک Actor (مثلاً «درخواست مرخصی»).	نگاه کنید به تفاوت Form/Request در بخش ۴.
Ticket	یک نوع خاص از Case که معمولاً برای رهگیری یک مسئله/درخواست خدمات با چرخه عمر مشخص (باز→در حال بررسی→بسته) استفاده می شود؛ در این پروژه معادل جدول tickets موجود است.	نگاه کنید به تفاوت Case/Ticket در بخش ۴.

۴. تفاوت های صریح خواسته شده

Workflow در برابر Process

Process طبقه‌بندی مفهومی «چه کاری» است (لایه‌ی Taxonomy، از APQC). **Workflow** پیاده‌سازی اجرایی «چگونه» است (گراف Nodeها). یک Process («تأیید مرخصی») می‌تواند در سازمان A یک Workflow تک‌مرحله‌ای و در سازمان B یک Workflow سه‌سطحی موازی باشد – هر دو همان Process را پیاده‌سازی می‌کنند اما Workflow متفاوتی دارند. **پیامد طراحی:** processes (Catalog/Taxonomy، بدون گراف) باید از workflow_templates (گراف قابل‌اجرا) جدا باشد؛ یک process می‌تواند صفر یا چند workflow_template پیوندشده داشته باشد (نگاه کنید به ۹۹).

Project در برابر Process

Process **تکرارپذیر و الگوار** است (همان مسیر بارها اجرا می‌شود: صدها «تیکت خرید»). Project **یک‌بارمصرف و هدف‌محور** است، حتی وقتی از یک Template ساخته می‌شود (دو پروژه‌ی «راه‌اندازی شعبه‌ی جدید» شبیه‌اند اما هرکدام Milestoneهای منحصر به فرد خودشان را دارند و با پایان پروژه، آن نمونه تمام می‌شود، نه تکرار). عملاً: Workflow Instance مربوط به یک Process معمولاً چند دقیقه تا چند هفته طول می‌کشد؛ Project معمولاً هفته‌ها تا ماه‌ها. **پیامد طراحی:** Task Manager/Project طبق ROADMAP.md موتور گردش کار جدا نمی‌سازد؛ بلکه از همین Workflow Engine برای «فرایندهای مدیریت پروژه» (مثل Change Request داخل پروژه) استفاده می‌کند، اما Projectها و Milestoneهای خودشان Entity جدا هستند، نه Instance گردش کار.

Task در برابر Activity

Activity سطح طبقه‌بندی (Level 4 در APQC) – «یک گام کلیدی» در سطح مفهومی، معمولاً چند Task را دربرمی‌گیرد و مستقیماً قابل‌اجرا نیست. **Task** یا (الف) ریزترین سطح (Level 5 APQC) Taxonomy یا (ب) در زمینه‌ی Runtime، یک Work Item مشخص که به یک نفر تخصیص داده شده. برای جلوگیری از ابهام، در این سند **Task همیشه یعنی واحد اجرایی قابل‌تخصیص (Runtime)**؛ برای سطح Taxonomy از «Taxonomy Task» استفاده می‌کنیم اگر لازم به تمایز باشد.

Case در برابر Ticket

هر دو Instance هستند که یک مسئله را ردیابی می‌کنند، اما **Ticket** معمولاً یک چرخه‌ی عمر **خطی و ساده** دارد (باز → در حال کار → بسته) و اغلب یک Actor مسئول اصلی دارد – دقیقاً مدل tickets فعلی پروژه. **Case** معمولاً **غیرخطی** است: می‌تواند چند زیرفرایند موازی داشته باشد، چند بار باز/بسته شود، و چند Actor با نقش‌های متفاوت هم‌زمان روی آن کار کنند (مثلاً یک پرونده‌ی بازرسی ایمنی که

تحقیقات، اقدام اصلاحی، و پیگیری حقوقی موازی دارد). **پیامد طراحی:** Ticket فعلی پروژه به عنوان یک نوع خاص و ساده از Case قابل مدل سازی است؛ Case Management عمومی تر می تواند در گام ۳ (موتور گردش کار) روی همان زیرساخت Work Item ساخته شود، بدون آنکه Ticket فعلی را بازنویسی کند.

Form در برابر Request

Form یک قالب ساختاریافته برای جمع آوری داده است (تعریف فیلدها، اعتبارسنجی، نوع) – چیزی شبیه `form_templates / form_fields` فعلی پروژه. **Request** عمل/رویداد ارسال یک Form توسط یک Actor برای شروع یک Process است. یک Form می تواند بارها توسط Request های مختلف پر شود؛ یک Request همیشه دقیقاً به یک نسخه ی مشخص از یک Form وابسته است (نگاه کنید به [۱۵-versioning](#) برای اینکه اگر Form عوض شود، Instance های قبلی چه می شوند).

Approval در برابر Decision

Approval یک نوع خاص و محدود از Decision است: یک Actor انسانی به یک گزینه ی از پیش تعریف شده (تأیید/رد، گاهی با توضیح) رأی می دهد و این رأی معمولاً یک Gate در مسیر اجراست. **Decision** مفهوم عمومی تر است: می تواند توسط یک انسان (Approval یک نمونه ی آن است)، یا توسط یک قانون خودکار (Business Rule/DMN Decision Table) گرفته شود، و لزوماً دو-گزینه ای هم نیست (می تواند بین N مسیر انتخاب کند). **پیامد طراحی:** `approvals / approval_steps` باید به عنوان یک نوع خاص از Work Item مدل شوند که خروجی اش یک Decision انسانی است؛ Decision های خودکار (Business Rule) نیازی به Work Item/Actor ندارند و مستقیماً توسط Rule Engine حل می شوند (نگاه کنید به [۱۴-rule-engine](#)).

Business Rule در برابر Workflow

Workflow توالی/شاخه بندی کنترل جریان را مشخص می کند («بعد از این، برو به آن»). **Business Rule** یک تصمیم مستقل و بدون حالت (Stateless) است که ورودی می گیرد و خروجی می دهد، بدون آنکه بداند در چه فرایندی صدا زده شده («اگر مبلغ < ۱۰۰ میلیون، تأییدکننده = مدیر»). طبق الگوی اثبات شده ی DMN/BPMN (نگاه کنید به [۱۴](#))، این دو باید به عنوان دو Entity کاملاً مجزا مدل شوند: یک Business Rule Task در Workflow فقط یک ارجاع (Reference) به یک Decision Table می دهد، نه اینکه منطق را در خود گراف Workflow Hardcode کند. این تفکیک یعنی: تغییر آستانه ی مبلغ تأیید نیازی به ویرایش گراف Workflow ندارد، فقط Decision Table عوض می شود.

۵. سایر مفاهیم (فهرست فشرده)

اصطلاح	تعریف کوتاه
Actor	هر موجودیتی (انسان یا سیستم) که می‌تواند یک Work Item را انجام دهد یا یک Event تولید کند.
Role	یک برچسب انتزاعی که Actorها را بر اساس مسئولیت گروه‌بندی می‌کند (نه یک شخص خاص) – پایه‌ی Assignment قانون‌محور (نگاه کنید به ۱۳).
Responsibility	پیوند بین یک Role/Actor و یک Work Item مشخص؛ می‌تواند «مسئول اجرا»، «مسئول تأیید»، یا «فقط مطلع (Informed)» باشد (الگوی RACI).
Input / Output	داده‌ای که یک Node/Activity قبل از اجرا نیاز دارد (Input) یا بعد از اجرا تولید می‌کند (Output) – پایه‌ی Variable Mapping بین Process اصلی و Sub-process (نگاه کنید به ۵A بخش Reusable Sub-process).
Artifact	هر خروجی ملموس یک Process (سند، رکورد، فایل) که ارزش مستقل از خود Instance دارد.
Document	یک نوع خاص از Artifact که محتوایش عمدتاً متنی/فایلی است و چرخه‌ی عمر بازبینی/تأیید/آرشیو دارد.
Record	ردیف داده‌ی ساختاریافته در یک جدول کسب‌وکاری (مثلاً یک <code>org_members</code> یا <code>ticket</code>) – Artifact لزوماً Record نیست (یک PDF امضا شده Record نیست) اما هر Record می‌تواند Artifact تولید کند.
Entity	اصطلاح عمومی برای هر «چیز» قابل‌مدل‌سازی در (Process، Form، Role، ...) Data Model – در این اسناد فقط برای اشاره به سطح داده استفاده می‌شود، نه سطح Runtime.
Dependency	رابطه‌ای که می‌گوید یک Node/Process نمی‌تواند شروع/تمام شود مگر Node/Process دیگری تمام شده باشد.
Exception	انحراف از مسیر طبیعی Workflow که نیازمند رسیدگی خارج از منطق پیش‌فرض است (خطا، شکست SLA، داده‌ی نامعتبر).
Escalation	نوع خاصی از واکنش به Exception یا نقض SLA: انتقال Work Item به یک Actor/Role با اختیار بالاتر. غیر-قطع‌کننده است (کار اصلی متوقف نمی‌شود؛ نگاه کنید به الگوی BPMN Escalation Event در ۵E).
Notification	پیام یک‌طرفه به یک Actor درباره‌ی رویدادی در Instance؛ برخلاف Work Item، نیازی به اقدام Actor برای ادامه‌ی Instance ندارد.

اصطلاح	تعریف کوتاه
Automation	هر Node/Activity که بدون دخالت انسانی توسط سیستم اجرا می‌شود (معادل BPMN Service/Script Task).
Event	وقوع یک چیز در زمان مشخص که می‌تواند یک Instance را شروع کند، متوقف کند، یا مسیرش را تغییر دهد (Message، Timer، Signal، Error – طبق واژگان BPMN، نگاه کنید به ۵۷).
Trigger	مکانیزم/شرطی که یک Event را به یک اقدام (شروع Instance، اجرای Node) وصل می‌کند.
SLA	تعهد زمانی روی یک Work Item/Instance («باید ظرف X ساعت پاسخ داده شود») – نگاه کنید به ۱۲.
KPI	معیار عددی برای سنجش عملکرد یک Process در طول زمان (نرخ تکمیل به موقع، میانگین زمان چرخه) – سطح تحلیلی/گزارشی، نه Runtime.

اسناد مرتبط: ۵۶-Process Engine · ۵۸-Workflow Node Catalog · ۵۷-Process Patterns · ۱۴-Rule Engine · ۵۹-Data Model Architecture

پایه: نسخه‌های صنعتی واقعی APQC (تأییدشده از منبع اصلی PCF)

Research نشان داد APQC دقیقاً ۱۹ نسخه‌ی صنعتی مستقل از PCF منتشر کرده (استخراج مستقیم از سند رسمی ۲۰۱۸، «Understanding the Elements of PCF»، APQC، و بازبینی‌شده با جست‌وجوی ۲۰۲۶). این‌ها صنایعی هستند که APQC عمق Level 3/4 اضافه‌ای برایشان منتشر کرده – نه یک اسکلت جدا، بلکه همان ۱۳ Category با شاخه‌های عمیق‌تر (نمونه‌ی تأییدشده: Process Group 4.3 در حالت عمومی ۴ Process دارد، در نسخه‌ی Aerospace & Defense به ۱۶ Process می‌رسد).

#	Industry (APQC-verified)	name_fa	apqc_variant_available
1	Aerospace & Defense	هوافضا و دفاعی	✓
2	Airline	هواپیمایی	✓
3	Automotive	خودروسازی	✓
4	Banking	بانکداری	✓
5	Broadcasting	پخش رادیویی/تلویزیونی	✓
6	City Government	شهرداری/دولت محلی	✓
7	Consumer Electronics	الکترونیک مصرفی	✓
8	Consumer Products	کالاهای مصرفی	✓
9	Corrosion (industry)	صنعت خوردگی	✓
10	Downstream Petroleum	پایین‌دستی نفت	✓
11	Education	آموزش	✓
12	Health Insurance Payor	بیمه‌گر سلامت	✓
13	Healthcare Provider	ارائه‌دهنده‌ی خدمات درمانی	✓
14	Insurance	بیمه	✓
15	Life Sciences	علوم زیستی/دارویی	✓

apqc_variant_available	name_fa	Industry (APQC-verified)	#
✓	خرده‌فروشی	Retail	16
✓	مخابرات	Telecommunications	17
✓	بالادستی نفت	Upstream Petroleum	18
✓	خدمات آب/برق/گاز	Utilities	19

سطح اطمینان: High – استخراج مستقیم از PDF رسمی APQC، نه خلاصه‌ی ثانویه.

گسترش: صنایع درخواستی کاربر که APQC نسخه‌ی مستقل ندارد

این صنایع طبق فهرست صریح کاربر (بخش چهارم Prompt) اضافه شدند؛ همه از **Cross-Industry PCF** به‌عنوان پایه استفاده می‌کنند (چون نسخه‌ی اختصاصی APQC ندارند)، اما به‌عنوان برچسب **industry** روی Organization معنادار و لازم‌اند تا فیلتر/پیشنهاد Process درست کار کند.

Industry	name_fa	زیرمجموعه‌ی نزدیک‌ترین APQC Industry (اگر هست)
Technology / Software / SaaS	فناوری/نرم‌افزار / سرویس ابری	نزدیک‌ترین: Consumer Electronics (سخت‌افزار) – برای نرم‌افزار خالص معادل مستقیم APQC نیست
Construction	ساخت‌وساز	–
Engineering	مهندسی	–
Architecture	معماری	–
Pharmaceutical	داروسازی	زیرمجموعه‌ی Life Sciences
Financial Services (غیربانکی)	خدمات مالی	نزدیک به Banking/Insurance اما مستقل
Retail / E-commerce	خرده‌فروشی/تجارت الکترونیک	Retail (APQC) پوشش می‌دهد؛ E-commerce زیرمجموعه
Logistics / Transportation / Warehousing	لجستیک / حمل‌ونقل/انبارداری	نزدیک‌ترین: Airline (فقط هوایی)؛ فاقد پوشش زمینی مستقل در APQC
Government / Municipality	دولت/شهرداری	City Government (APQC) فقط سطح شهرداری را پوشش می‌دهد؛ دولت ملی مستقل نیست

Industry	name_fa	زیرمجموعه‌ی نزدیک‌ترین APQC Industry (اگر هست)
Energy / Oil & Gas / Mining	انرژی/نفت‌وگاز/ معادن	Upstream/Downstream Petroleum (APQC) نفت را پوشش می‌دهد؛ معدن مستقل نیست
Agriculture / Food	کشاورزی/غذایی	نزدیک به Consumer Products
Hospitality / Travel	مهمان‌نوازی/ گردشگری	—
Real Estate / Property Management	املاک/مدیریت ساختمان	—
Media / Advertising	رسانه/تبلیغات	نزدیک به Broadcasting (فقط پخش)
Consulting / Professional Services	مشاوره/خدمات حرفه‌ای	—
Legal Services	خدمات حقوقی	—
Security Services	خدمات امنیتی	—
Nonprofit	غیرانتفاعی	—

نتیجه‌ی طراحی: فیلد `industries.apqc_variant_available` این تفاوت را در Data Model کدگذاری می‌کند (نگاه کنید ۹۵) – برای صنایع بدون نسخه‌ی اختصاصی، محصول باید صریحاً Cross-Industry Process پیشنهاد بدهد، نه وانمود کند نسخه‌ی تخصصی وجود دارد.

قانون طلایی: صنعت یک لایه‌ی Taxonomy نیست، یک برچسب فیلترکننده است

طبق یافته‌ی Research (بخش ۵ گزارش APQC): «industry variants share the same root» در Data Model به `processes` مستقیم متصل نمی‌شود با یک ستون `industry_id` ساده؛ به‌جایش جدول `process_industries(process_id, industry_id, relevance)` استفاده می‌شود (نگاه کنید ۹۵) – چون اکثر Process‌ها (مثل «Leave Request») در همه‌ی صنایع یکسان‌اند و فقط تعداد کمی (مثل «Vendor Onboarding» در Downstream Petroleum) واقعاً صنعت-محور می‌شوند.

[Domain Taxonomy](#) · [٥-Process Taxonomy](#) · `data/process-٤-` **اسناد مرتبط:** `library/industries.json`

۰۴ – Cross-Functional و طراحی Domain Taxonomy

پایه: ۱۳ Category سطح یک APQC (تأییدشده از منبع اصلی، نسخه ی ۷.3.1،
سپتامبر ۲۰۲۳)

کد	نام (نسخه ی ۲۰۲۳)	name_fa	نوع
1.0	Develop Vision and Strategy	تدوین چشم انداز و استراتژی	Operating
2.0	Develop and Manage Products and Services	توسعه و مدیریت محصولات / خدمات	Operating
3.0	Market and Sell Products and Services	بازاریابی و فروش	Operating
4.0	Manage Supply Chain for Physical Products	مدیریت زنجیره ی تأمین محصولات فیزیکی	Operating
5.0	Deliver Services	تحویل خدمات	Operating
6.0	Manage Customer Service	مدیریت خدمات مشتری	Operating
7.0	Develop and Manage Human Capital	توسعه و مدیریت سرمایه ی انسانی	Management & Support
8.0	Manage Information Technology (IT)	مدیریت فناوری اطلاعات	Management & Support
9.0	Manage Financial Resources	مدیریت منابع مالی	Management & Support
10.0	Acquire, Construct, and Manage Assets	تحصیل، ساخت و مدیریت دارایی	Management & Support
11.0	Manage Enterprise Risk, Compliance, Remediation, and Resiliency	مدیریت ریسک، انطباق و تاب آوری	Management & Support
12.0	Manage External Relationships	مدیریت روابط بیرونی	Management & Support

کد	نام (نسخه‌ی ۲۰۲۳)	name_fa	نوع
13.0	Develop and Manage Business Capabilities	توسعه و مدیریت قابلیت‌های کسب‌وکار	Management & Support

نکته‌ی اطمینان: تفکیک (1-8)/Management&Support(9-13) در بالا با اطمینان **Medium** علامت‌گذاری شده – در متن اصلی APQC این مرز به این وضوح بیان نشده، از یک منبع ثانویه (Due Process Consulting) گرفته شده و صرفاً برای درک شهودی آورده شده، نه یک قاعده‌ی رسمی APQC.

نگاشت به Domain‌های درخواستی کاربر (بخش چهارم Prompt)

Domain‌های عمومی/Cross-Industry که کاربر لیست کرده بود، همگی زیرمجموعه‌ی یکی از ۱۳ Category بالا قرار می‌گیرند – هیچ **Domain جدیدی** نیاز به **Category جدید** نداشت، این خودش تأییدی بر جامعیت APQC است:

Domain (کاربر)	زیرمجموعه‌ی کدام Category
Management, Strategy	1.0
Operations	2.0, 5.0 (بسته به نوع خروجی)
Sales, Marketing	3.0
Supply Chain, Procurement	4.0
Customer Service	6.0
HR	7.0
IT	8.0
Finance, Accounting	9.0
Facilities	10.0
Legal, Compliance, Quality, Security	11.0
Administration	پراکنده (9.0/13.0 بسته به فعالیت)

Domain (کاربر)	زیرمجموعه‌ی کدام Category
Document Management, Knowledge Management	13.0 (نزدیک‌ترین معادل APQC؛ APQC آن را به عنوان Capability عمومی می‌بیند نه یک Category مستقل)
Project Management	پراکنده در همه‌ی Category ها – طبق تعریف Value Stream در ۵۲ ، بیشتر شبیه یک Value Stream عمودی است تا یک Domain افقی

چرا Taxonomy فقط بر اساس ساختار سازمانی کافی نیست (پاسخ صریح به هشدار کاربر)

مثال دقیقاً همان مثالی که کاربر آورد – «Purchase Request → Manager Approval → Procurement Review → Vendor Selection → PO → Delivery → Invoice Matching → Payment» – طبق Research واقعی (سند [۵۵](#))، این فرایند واحدهای زیر را قطع می‌کند:

Requester (هر دپارتمانی) → 4.0/9.0 (Procurement/Finance) → 9.0 (Finance، Invoice Matching) → 9.0 (Payment)

اگر Taxonomy فقط بر مبنای Department طراحی شود، این فرایند باید به یک واحد «تعلق» داشته باشد که غلط است – عملاً در Procurement زندگی می‌کند اما Approval از دپارتمان درخواست‌دهنده شروع می‌شود و در Finance تمام می‌شود. **راه‌حل معماری:**

processes.domain_id نشان‌دهنده‌ی «کجا Owner می‌شود» است (Home Domain)، نه «چه کسانی درگیرند». درگیری واقعی چند-دپارتمانی از طریق:

1. **process_industries / تگ‌ها** برای دسته‌بندی افقی اضافی

2. **خود گراف Workflow** (نه Taxonomy) – که واقعاً نشان می‌دهد کدام Node به کدام

Actor/Role/Department تعلق دارد (دقیقاً معادل BPMN Pool/Lane، تأییدشده در Research:

«Pool = participant container، Lane = زیرمجموعه‌ی نقش/دپارتمان درون یک Pool»)

نتیجه: Cross-Functional بودن در سطح **اجرا** (Lane در گراف Workflow) حل می‌شود، نه در سطح Taxonomy. Taxonomy فقط برای پیدا کردن/دسته‌بندی Process در Library است؛ واقعیت چندواحدی بودن اجرا در گراف زندگی می‌کند.

Function (سطح دوم – Process Group)

هر Domain به چند Function تقسیم می‌شود (معادل APQC Process Group، سطح دو). نمونه برای Domain «Finance» (Category 9.0):

- Accounts Payable (≈ APQC 9.2)
- Accounts Receivable / Revenue Accounting (≈ APQC 9.3)
- Financial Planning & Budgeting (≈ APQC 9.1)
- Financial Close & Reporting

فهرست کامل Function per Domain در `data/process-library/domains.json` آمده (به‌همراه `functions` تلویحی در Slug هر Process، نه یک فایل جدا – طبق تصمیم در [data--۰۹](#) `model` که Function ها را به همان جدول Process Group متصل نگه می‌دارد تا حجم فایل‌های JSON غیرضروری زیاد نشود).

اسناد مرتبط: [data/process--۰۳](#) · [۰۵-Process Taxonomy](#) · [Industry Taxonomy](#)
`library/domains.json`

۵. Process Taxonomy (فرایندهای واقعی هر حوزه)

این سند خلاصه‌ی خواناست؛ جزئیات کامل (Trigger/Actors/Steps/Forms/SLA برای هر Process) در `data/process-library/processes.json` و نسخه‌ی کاملاً بازشده‌ی ۲۰ فرایند در `data/process-library/sample-full-processes.json` است. همه‌ی موارد زیر از منابع واقعی Vendor استخراج شدند (نه حدس) – منبع هرکدام در JSON ثبت شده.

Process Type Taxonomy ثانویه (طبقه‌بندی بر اساس نوع، نه صنعت/دامنه)

طبق درخواست صریح کاربر، فرایندها علاوه بر Domain، بر اساس **نوع رفتاری** هم دسته‌بندی می‌شوند – این محور دوم است که به Builder اجازه می‌دهد الگوی مشترک بین Domain‌های مختلف را تشخیص دهد:

Process Type	تعریف کوتاه	نمونه از Research
Request	یک Actor چیزی می‌خواهد	Leave Request, Asset Request
Approval	تصمیم دودویی/چندگزینه‌ای روی یک درخواست	Purchase Requisition approval
Review	بازبینی بدون تصمیم قطعی	Contract Review (پیش از Approval نهایی)
Service	ارائه‌ی یک خدمت مشخص به یک متقاضی	IT Service Request Fulfillment
Incident	واکنش به یک اختلال غیرمنتظره	ITSM Incident Management
Case Management	پیگیری غیرخطی یک پرونده	Security Incident Response, Complaint Handling
Compliance	اطمینان از رعایت یک قاعده/قانون	Policy Exception Request
Audit	بررسی سیستماتیک و مستندسازی یافته	Audit Finding Remediation
Procurement	تحصیل کالا/خدمت از بیرون سازمان	Purchase Requisition, RFQ
Financial	جابه‌جایی/کنترل منابع مالی	Invoice Approval, Budget Approval
HR	مرتبط با چرخه‌ی عمر کارمند	Onboarding, Performance Review

نمونه از Research	تعریف کوتاه	Process Type
Document Approval Cycle	چرخه‌ی عمر یک سند	Document
Project Change Request	فعالیت‌های محدود به یک هدف یک بار مصرف	Project
Refund Request, CSAT Follow-up	تعامل مستقیم با مشتری بیرونی	Customer Service
Maintenance/Repair Request	نگهداری دارایی فیزیکی/دیجیتال	Maintenance
Asset Inspection	بازرسی دوره‌ای	Inspection
Shift Change/Swap	تخصیص زمان/منبع	Scheduling
Employee/Customer/Vendor Onboarding	ورود یک Entity جدید (کارمند/مشتری/ تأمین‌کننده)	Onboarding
Employee Offboarding	خروج یک Entity	Offboarding
(جزء اکثر Process‌های بالا، نه یک Process مستقل معمولاً)	انتقال به سطح بالاتر مسئولیت	Escalation
Audit Finding, Change-of-plan	انحراف از مسیر عادی	Exception
Contract Renewal	تمدید یک تعهد/قرارداد رو به انقضا	Renewal
Vendor Evaluation, Performance Review	ارزیابی کیفی/کمی	Assessment
Vendor Onboarding (بخش ثبت)	ثبت اولیه‌ی یک Entity در سیستم	Registration
Service Request Fulfillment	تکمیل عملیاتی یک درخواست پذیرفته‌شده	Fulfillment

خلاصه‌ی فرایندهای واقعی به تفکیک Domain

HR (Category 7.0)

Employee Onboarding · Employee Offboarding · Leave/Time-off Request · Recruitment (Requisition-to-Hire) · Performance Review · Training Request · Expense Reimbursement · Shift Change/Swap · Overtime Approval · Promotion/Transfer · Disciplinary Action

منابع: Workday, BambooHR, SAP SuccessFactors, SAP Concur, Oracle HCM (جزئیات کامل هرکدام در processes.json).

IT Service Management (Category 8.0)

Incident Management · Problem Management · Change Management
Service (Normal/Standard/Emergency) – سه Process Type متفاوت با یک والد مشترک) · Request Fulfillment · Access Request/Provisioning · Asset Request · Software Request · IT Onboarding/Offboarding · Security Incident Response

منابع: ITIL 4، ServiceNow، (Practice، ۳۴ مورد Service Management).

یافته‌ی مهم APQC PCF **Cross-check:** در Category 8.0 این فرایندها را به صورت جداگانه نام‌گذاری نمی‌کند – فقط زیر یک Process با نام «Operate IT user support» چند Activity کلی دارد (Triage/Manage/Escalate/Resolve). این یعنی برای **ITIL**، Domain IT، **باید منبع اصلی نام‌گذاری/ساختار باشد، نه APQC** – دقیقاً طبق قانون کاربر «از یک منبع به عنوان حقیقت مطلق استفاده نکن».

Procurement / Supply Chain (Category 4.0)

RFQ/RFP · Purchase Requisition (مترادف: Purchase Request, Procurement Request) · Vendor/Supplier Onboarding · Vendor Evaluation · Purchase Order Creation · Goods Receipt · Three-way Invoice Matching · Payment Request · Contract Renewal

منابع: SAP Ariba, Coupa, Oracle Procurement Cloud.

Finance/Accounting (Category 9.0)

Expense Claim · Invoice Approval/Accounts Payable (همان Process با Expense Deduplication در Reimbursement – **Duplicate** HR شناسایی‌شده و **Merge** شده، نگاه کنید بخش Deduplication پایین) · Budget Approval · Financial Close · Order-to-Cash/Accounts Receivable-Collections

منابع: SAP، NetSuite.

Sales/CRM (Category 3.0)

Lead-to-Opportunity · Quote/Proposal Approval · Deal Desk Approval · Contract Approval
Customer Onboarding · (همپوشان با Legal – نگاه کنید پایین)
منابع: Salesforce، HubSpot.

Customer Service (Category 6.0)

Ticket Escalation · CSAT Follow-up · Refund/Return Request · Customer Complaint Handling (مستندسازی کم عمق – نگاه کنید [gap-analysis-۱۶](#))
منابع: Zendesk، Salesforce Service Cloud.

Legal/Compliance (Category 11.0)

Compliance · (Document Sign-off با همپوشان) Contract Review/Approval · NDA Request
Audit Finding Remediation · Policy Exception Request
منابع: DocuSign CLM، Info-Tech Research، Section508.gov (نمونه‌ی واقعی دولت فدرال آمریکا).

Facilities/Operations (Category 10.0)

SLA با OSHA، استاندارد Maintenance/Repair Request (Work Order) · Incident/Safety Report
رسمی: فوت=۸ ساعت، بستری/قطع عضو=۲۴ ساعت، سایر=۷ روز) Facility Access Request · (مستندسازی کم –
الگوی Request-to-Fulfillment عمومی) Asset Inspection · (مستندسازی کم –
الگوی CMMS عمومی)

Document Management (Category 13.0)

Digital Signature/Sign-off · (Parallel یا Sequential) Document Approval/Review Cycle
Workflow
منابع: SharePoint، M-Files، DocuSign.

Synonym و Deduplication (طبق قانون بخش ۲۶ کاربر)

Research صراحتاً موارد زیر را به عنوان همان مفهوم با نام‌های مختلف شناسایی کرد – این‌ها یک
process با چند process_synonyms می‌شوند، نه چند Process جدا:

Synonyms یافت شده	Canonical Name
Purchase Request, Procurement Request	Purchase Requisition
Expense Claim	Expense Reimbursement
Termination, Exit Process	Employee Offboarding
Recruit-to-Hire زیرمجموعه‌ی Recruitment است، نه مترادف Onboarding – این دو عمداً جدا نگه داشته شدند چون Recruitment قبل از استخدام و Onboarding بعد از آن است	Onboarding (عمومی)
زیرمجموعه‌ی عملیاتی Document Sign-off Pattern؛ به‌عنوان Process مستقل نگه داشته شد چون Trigger/Actor خاص خودش را دارد، اما در related_processes به Document Approval لینک شد	NDA Request
این دو Merge نشدند – هرچند شبیه‌اند، Domain متفاوت (Sales در برابر Legal) و معمولاً Actor اول متفاوت (Deal Desk در برابر Legal Counsel) دارند؛ در related_processes به هم لینک شدند تا در ال به کاربر «آیا منظورتان این است؟» نشان داده شود	Contract Approval (Sales) در برابر Contract Review/Approval (Legal)

اسناد مرتبط: ۰۶- data/process-library/processes.json · [۱۶-Gap Analysis](#) · [۱۰-Process Library](#) · [Process Patterns](#)

۰۶ – Process Patterns (الگوهای تکرارشونده)

منبع اصلی: تحلیل Cross-Domain در Research فرایندهای واقعی (سند ۵هـ) که صراحتاً نشان داد اکثر فرایندهای سازمانی از تعداد محدودی الگو ساخته می‌شوند – نه اینکه هر Domain الگوی خاص خودش را داشته باشد.

الگوهای اصلی (۱۴ مورد، هرکدام با نمونه‌ی واقعی از Research)

۱. Simple Approval

Request → Review → Approval → Execution → Closure

نمونه‌ی واقعی: Leave Request (SAP SuccessFactors) – تک‌سطحی، یک Approver.

۲. Multi-Level (Sequential) Approval

Request → Supervisor → Manager → Director → Final Approval → Execution

نمونه‌ی واقعی: «Concur Expense» Management Chain Routing – بالا می‌رود تا به Approver ای با سقف اختیار کافی برسد (Escalate-by-authority-limit، نه صرفاً سلسله‌مراتب ثابت).

۳. Parallel Approval

Request → {Finance, Legal, Technical} (هم‌زمان) → Merge → Final Decision

نمونه‌ی واقعی: «Salesforce CPQ Advanced Approvals» Parallel simultaneous approvers.

۴. Threshold-Based Conditional Approval

Request → Decision(amount) → [amount > X → Director] یا [amount ≤ X → Manager] →

Execution

نمونه‌ی واقعی: NetSuite AP – فاکتور بالای ۲۰,۰۰۰ دلار به CFO Escalate می‌شود؛ این عملاً ترکیب Branch + Escalation Pattern است، نمونه‌ی مستقیم DMN Decision Table (if amount > X then «approver = director» نگاه کنید [۱۴](#)).

۵. Sequential Review-and-Rework

Submission → Review → (Changes Requested) → Rework → Resubmission → Review → Approval

نمونه‌ی واقعی: DocuSign CLM Contract Review – با Deadline و یادآوری خودکار هر دور.

۶. SLA / Escalation Chain

Request → Assignment → SLA Timer شروع → [Breach نزدیک → Reminder] → [Breach → Escalate L1] → [Breach ادامه‌ی → Escalate L2] → Resolution

نمونه‌ی واقعی: ServiceNow Major Incident + Zendesk «Hours since SLA breach» Automation – سه پیاده‌سازی متفاوت واقعی برای همین الگو در Research پیدا شد (نگاه کنید [۱۵](#)) – این الگو مهم‌ترین الگوی غیر-Approval کل Research است.

۷. Exception Handling (Non-Interrupting)

Process (در حال اجرا) → [Exception می‌دهد] → Investigation → Resolution → Resume همان Process

نمونه‌ی واقعی: BPMN Non-interrupting Boundary Event – کار اصلی متوقف نمی‌شود، یک مسیر موازی رسیدگی باز می‌شود.

۸. Conditional Branch

Request → Decision → Path A یا Path B

پایه ی فنی: BPMN Exclusive Gateway / Step Functions Choice / n8n Switch – سه پیاده سازی مستقل همین یک مفهوم را با واژگان متفاوت (Gateway/Choice/Switch) دارند؛ شواهد قوی که این یک الگوی جهانی ست، نه سلیقه ی یک Vendor.

۹. Loop (Rework Cycle)

Submit → Review → Rework → Review → ... → Approval (با سقف تکرار)

تفاوت با #۵: اینجا تأکید روی محدودیت تکرار (جلوگیری از Infinite Loop) است، نه صرفاً یک بار برگشت.

۱۰. Parallel Independent Tasks (Fork/Join)

Process → {Task A, Task B, Task C} (مستقل از هم) → Join → Continue

نمونه ی واقعی: IT Onboarding (ServiceNow EOT) – فراهم سازی سخت افزار، ایجاد حساب، آموزش امنیتی هم زمان اجرا می شوند، بدون وابستگی به هم.

۱۱. Delegation / Temporary Reassignment

Task → Assigned User (غایب) → Delegate → Task تکمیل می شود توسط Delegate

نمونه ی واقعی: «Delegated Approver» «Salesforce Approval Processes» (نمی تواند دوباره Reassign کند، فقط تصمیم می گیرد) + BPM Deadline-based Reassignment.

۱۲. Scheduled / Recurring Process

Date/Event رخ می دهد → Trigger → Work Item ساخته می شود → Execute → Close

نمونه ی واقعی: «۱» – Contract Renewal (Oracle) ماه قبل از انقضا «Reminder خودکار می سازد.

۱۳. Generic Request-to-Fulfillment

Submit Request (Form) → [Approval اختیاری] → Route to Fulfiller → Provision/Process
→ Notify Requester → Confirm/Close

پرتکرارترین الگوی یافت‌شده در کل Research – عیناً در Facility، ServiceNow Service Catalog، Actor/Object فقط Maintenance Work Order، Training Enrollment، NDA Request عوض می‌شود.

۱۴. Generic Onboarding/Offboarding Orchestration

Triggering Event → Fan-out به چند Department (چکلیست موازی) → Provisioning →
Milestone Tracking → Completion State

نمونه‌ی واقعی: تقریباً عین‌هم در HR Onboarding (Workday)، IT Onboarding (ServiceNow)، Customer Onboarding (HubSpot)، Vendor Onboarding (Coupa) – این الگو ترکیبی از ۱۰# (Parallel Tasks) + یک Case-level Status Machine است، نه یک Pattern کاملاً جدید؛ اما به‌قدری پرتکرار است که ارزش Template مستقل دارد.

نکته‌ی معماری: چرا این الگوها مهم‌اند

هر الگو در بالا باید در `workflow_patterns.json` به‌عنوان یک **Template** قابل درگ‌ودرآپ در **Builder** موجود باشد (نه فقط مستند). یعنی کاربر وقتی «Approval Node» را می‌گذارد، **Builder** باید پیشنهاد بدهد «می‌خواهید این را Multi-Level کنید؟» – دقیقاً چون ۱# و ۲# در عمل یک الگو با یک پارامتر (تعداد سطح) هستند، نه دو Node متفاوت.

رابطه‌ی الگو با Node Catalog: هر الگو ترکیبی از Node‌های ۷# است، نه یک Node جدید. مثلاً الگوی ۶#
`Form + = ۱۳# الگوی Wait/Timer + Escalation + Approval = (SLA/Escalation)`
`Approval (اختیاری) + Send Notification + Create Record`.

اسناد مرتبط: ۵-۱۲-SLA · ۷-Node Catalog · Process Taxonomy

۷.۰ – کاتالوگ Node های Workflow Builder

منبع: مقایسه‌ی مستقیم واژگان BPMN 2.0 (استاندارد OMG) با پنج موتور واقعی – Camunda، AWS – Input/Output، هدف، Node: برای هر Step Functions، n8n، ServiceNow Flow Designer، Pega Configuration، وضعیت‌های ممکن، اعتبارسنجی، مجوز، مدیریت خطا، نیاز به Audit – طبق درخواست صریح کاربر.

اصل طراحی: چهار دسته‌ی بالادستی

هر Node دقیقاً در یکی از این چهار دسته قرار می‌گیرد (این دسته‌بندی از تلفیق BPMN Flow-Object-types با واژگان n8n/Step Functions به دست آمد):

1. **Flow Control** – مسیر اجرا را تعیین می‌کند، خودش کار کسب‌وکاری انجام نمی‌دهد (Start/End/Gateway/Wait/Loop/Merge)
2. **Human Work** – منتظر یک Actor انسانی می‌ماند (Task/Approval/Review/Form)
3. **Automation** – بدون انسان اجرا می‌شود (Create/Update Record، Send Notification، Webhook، Script)
4. **Sub-graph** – یک گراف دیگر را فرامی‌خواند (Subprocess/Call Process)

جدول کامل

Node	دسته	هدف	Input	Output
Start	Flow Control	نقطه‌ی ورود گراف؛ می‌تواند دستی، زمان‌بندی‌شده، یا رویدادمحور باشد (طبق BPMN Start Event: (None/Message/Timer/Signal/Conditional	Trigger payload	متغیرهای اولیه؛ Instance
End	Flow Control	پایان یک مسیر؛ می‌تواند End موفق یا End شکست باشد (BPMN: (None/Error/Terminate End	–	نتیجه‌ی نهایی Instance

Node	دسته	هدف	Input	Output
Task	Human Work	یک واحد کار دستی که موتور فقط تکمیلش را ثبت می‌کند، منتظر جزئیات نمی‌ماند (معادل BPMN Manual Task)	متن دستورالعمل	تیک تکمیل
Approval	Human Work	تصمیم Approve/Reject توسط یک یا چند Actor؛ اولین شهروند در Pega/ServiceNow، نه یک Task دستی معمولی	Context (رکورد/ Form مرتبط)	decision: roved rejected + note
Review	Human Work	بازبینی بدون تصمیم دودویی؛ خروجی می‌تواند Approve/Request-Changes/Comment باشد (شبیه چرخه Sequential Review)	سند/ داده‌ی مرور	نظرات + وضعیت
Form	Human Work	جمع‌آوری داده‌ی ساختاریافته از یک Actor؛ خروجی مستقیماً process_variables می‌شود	form_version_id	داده‌ی پرشده
Decision / Condition	Flow Control	مسیر را بر اساس داده تعیین می‌کند؛ خودکار، بدون Actor – معادل BPMN Exclusive Gateway ساده یا Business Rule Task اگر به Decision Table وصل شود	متغیرهای Instance	مسیر انتخابی
Branch (Exclusive)	Flow Control	دقیقاً یک مسیر از N مسیر انتخاب می‌شود (BPMN Exclusive/XOR Gateway)	نتیجه‌ی Decision قبلی	یک مسیر خروج
Parallel (Fork)	Flow Control	چند مسیر همزمان باز می‌شوند (BPMN Parallel/AND Gateway)	—	N مسیر فعال
Merge (Join)	Flow Control	مسیرهای موازی به یک مسیر برمی‌گردند؛ منتظر همه یا حداقل تعداد مشخص می‌ماند	N مسیر ورودی	یک مسیر خروج

Node	دسته	هدف	Input	Output
Wait / Delay	Flow Control	مکث تا زمان مشخص یا مدت مشخص (BPMN Timer Intermediate Event)	–	ادامه‌ی مسیر
Timer / Reminder	Flow Control	مشابه Wait اما غیرقطع‌کننده – کنار یک Human Node می‌نشیند و بدون توقف کار اصلی هشدار می‌دهد (BPMN Non-interrupting Timer Boundary Event)	–	Notification
Escalation	Flow Control	تغییر Actor/سطح مسئولیت یک Work Item بدون لغو آن (BPMN Escalation Event)	Work Item فعلی	Work Item با Assignee/Role جدید
Loop	Flow Control	تکرار یک زیرمسیر تا شرط خروج (BPMN Standard Loop)	شرط ادامه	–
Multi-Instance (Map)	Flow Control	همان زیرگراف روی هر عضو یک آرایه اجرا می‌شود (معادل BPMN Map / Step Functions Multi-Instance marker)	آرایه‌ی ورودی	آرایه‌ی نتایج
Subprocess / Call Process	Sub-graph	فراخوانی یک workflow_template دیگر (نگاه کنید ه‌ا)	Contract ورودی تعریف‌شده	Contract خروج تعریف‌شده
Create Record /	Automation	ساخت یا ویرایش یک رکورد در یکی از ماژول‌های محصول (Ticket، Task، Project، ...)	فیلدهای رکورد	شناسه‌ی رکورد ساخته/ویرایش‌شده

Node	دسته	هدف	Input	Output
Update Record				
Create Task / Create Ticket / Create Project	Automation	نمونه‌ی خاص Create Record برای ماژول‌های اصلی محصول (نگاه کنید ۱۱-integration)	—	شناسه‌ی رکورد ساخته‌شده در ماژول مقصد
Send Email / Send SMS / In-app Notification	Automation	اطلاع‌رسانی یک‌طرفه (نگاه کنید Notification در ۵۲)	گیرنده + Template پیام	رسید ارسال
Webhook / API Call / Integration	Automation	فراخوانی یک سیستم بیرونی (معادل BPMN (Service Task	URL/Payload	Response
Script / Function	Automation	منطق سفارشی کوتاه (معادل BPMN Script Task) — باید محدود و Sandbox شود، نه دسترسی آزاد به کل سیستم	متغیرهای Instance	متغیرهای جدید تغییر یافته
Manual Action	Human Work	معادل Task ولی بدون هیچ ردیابی جزئیات (BPMN Manual Task) — مثل «تماس تلفنی بگیر»	—	تیک تکمیل

قاعده‌ی طراحی مهم: Approval یک Node مجزا از Task است

طبق یافته‌ی Research «Approve/Reject» Pega: به‌عنوان Step Type آماده، نه منطق دستی؛
ServiceNow: Approval به‌عنوان Action-Category (مجزا)، **Approval** نباید صرفاً یک Task با یک
Checkbox باشد – باید از ابتدا Node مستقل با مفاهیم Multi-level/Parallel/Threshold باشد، چون
این دقیقاً پرتکرارترین Pattern در کل Research بود (نگاه کنید [۶-۰۶-patterns](#)).

اسناد مرتبط: [۶-۰۶-Rule Engine](#) · [۸-۰۸-Engine Architecture](#) · [Process Patterns](#)

۰۸ – معماری پیشنهادی Process Engine

معماری لایه‌ای اصلی

پیشنهاد اولیه‌ی کاربر (Workflow → Workflow Definition → Process Definition → Version → Workflow Instance → Work Items → Tasks/Approvals/Forms/Events → Completion/Exception/Research) تأیید می‌شود، با یک اصلاح مهم: بر اساس بررسی مستقیم Temporal، Camunda 7/8، و AWS Step Functions (سند [۱۵-versioning](#))، هر سه موتور واقعی «Definition» و «Version» را عملاً یک خانواده از رکوردها می‌دانند (هر Version یک Snapshot غیرقابل‌تغییر از همان Definition است)، نه دو لایه‌ی مجزا در توالی عمودی. پس معماری اصلاح‌شده:

Process (Taxonomy – از APQC/Library، بدون گراف اجرایی)

| پیاده‌سازی ۰..N



Workflow Template (در Camunda 8 bpmnProcessId شناسه‌ی پایدار، معادل)

| هر انتشار یک نسخه‌ی غیرقابل‌تغییر می‌سازد



Workflow Version (شماره‌ی صحیح صعودی – این خود Node/Edge گراف Snapshot)

(اجراشدنی است "Definition")

| (Template نه به) هر شروع، دقیقاً به یک نسخه‌ی مشخص گره می‌خورد



Workflow Instance (یک اجرای واقعی و زمان‌دار، وضعیت + متغیرها)

| اجراشونده یک یا چند مورد زیر تولید می‌کند Node هر



Work Items (واحد قابل‌تخصیص یا – Task / Approval / Form / Event / Automation)

(رویداد)



Completion یا Escalation یا Exception (نه لزوماً خروج از Work Item، خروج از Instance)

|

▼

Instance Completion (Success) | Instance Terminated (Cancelled/Failed)

چرا این اصلاح مهم است: اگر «Workflow Definition» و «Workflow Version» دو جدول جدا با رابطه‌ی یک‌به‌چند باشند، تغییر می‌کند به دو جدول با معنای تقریباً یکسان و ریسک ناهم‌گامی. مدل صحیح (که هر سه موتور بررسی‌شده استفاده می‌کنند): `workflow_templates` فقط شناسه‌ی پایدار + Metadata نگه می‌دارد؛ خود گراف در `workflow_versions` است و هرگز In-place ویرایش نمی‌شود – نسخه‌ی جدید یعنی ردیف جدید.

چرا نه Definition-محور صرف (BPMN) و نه Code-محور صرف (Temporal)

Research دو الگوی رقیب پیدا کرد:

- **BPMN/Camunda (Graph-defined):** گراف Node/Edge داده‌محور، مناسب برای کاربر غیرتوسعه‌دهنده که با Builder بصری کار می‌کند.
- **Temporal (Code-defined):** Workflow = کد، اجرا = Durable Execution با Replay قطعی؛ برای این نوع پروژه نامناسب چون **کاربر نهایی محصول ما مدیر سازمان است، نه توسعه‌دهنده** – نمی‌تواند کد بنویسد.

تصمیم: معماری Graph-defined (شبیه BPMN/Camunda) پایه قرار می‌گیرد، چون تنها الگویی است که یک Builder بصری/غیرکدنویسی روی آن قابل‌ساخت است – دقیقاً همان دلیلی که n8n، Power Automate، ServiceNow Flow Designer و Zapier (همه در دسته‌ی ابزار کاربر-نهایی) هم از الگوی Node/Edge استفاده می‌کنند نه الگوی کد.

مرز Template در برابر Runtime

لايه	شامل چه چیزی؟	تغییرپذیر؟
Template (<code>workflow_templates</code> + <code>workflow_versions</code>)	گراف Node/Edge، Form های متصل، Rule های ارجاع شده، SLA	فقط با انتشار نسخه‌ی جدید؛ نسخه‌ی منتشرشده هرگز In-place ویرایش

لايه	شامل چه چيزي؟	تغييرپذير؟
	پيش فرض، نقش هاي Assignment	نمي شود (طبق الگوي هر سه موتور بررسي شده)
Runtime (workflow_instances + work_items + (process_variables	وضعيت واقعي، مقادير واقعي متغيرها، Actor واقعي هر Work Item، Timestamp	دائماً در حال تغيير تا لحظه ي Completion

قانون طلایی (مستخرج از هر سه موتور بررسی شده): یک **Instance** که قبلاً شروع شده، به نسخه‌ای که با آن شروع شده «سنجاق (Pin)» می‌ماند و با انتشار نسخه‌ی جدید خودکار منتقل نمی‌شود؛ انتقال (Migration) همیشه یک عملیات صریح و جداگانه است (جزئیات در [۱۵- versioning](#)).

Reusable Sub-process (پاسخ به الزام «Manager Approval قابل استفاده در چند Process»)

دو الگوی متفاوت و هر دو معتبر پیدا شد (نگاه کنید به Research BPMN):

1. **BPMN Call Activity (الگوی Camunda)**: فراخوانی یک Process کاملاً مستقل که به تنهایی هم قابل اجراست، اما Process دیگری هم می‌تواند آن را صدا بزند. Parent در محل Call Activity مسدود (Block) می‌شود تا Child کامل شود؛ داده با in / out Mapping (نگاشت متغیر به متغیر) رد و بدل می‌شود، پیش فرض: هم نام بدون Mapping صریح منتقل می‌شود.
2. **ServiceNow Subflow (الگوی Function-like)**: یک واحد فقط-قابل فراخوانی که خودش هیچ Trigger مستقلی ندارد – دقیقاً مثل یک تابع: Typed Input های می‌گیرد، Typed Output های برمی‌گرداند.

تصمیم برای این محصول: هر دو الگو را با یک Flag پشتیبانی کنید: `workflow_templates.is_callable_only` boolean. اگر `true`، این Template در لیست «شروع مستقل» به کاربر نشان داده نمی‌شود و فقط به عنوان یک Node از نوع `Subprocess / Call` در Process Template های دیگر قابل ارجاع است. «Manager Approval» به طور طبیعی `is_callable_only = true` خواهد بود؛ اما مثلاً «Purchase Request» که هم مستقل هم به عنوان زیرفرایند در «Vendor Onboarding» قابل فراخوانی است، `false` می‌ماند – این دقیقاً همان انعطاف BPMN Call Activity است.

Input/Output هر Sub-process با یک Contract صریح تعریف می‌شود (شبیه ServiceNow، نه Mapping آزاد Camunda، چون برای کاربر غیرفنی خواناتر است):

```
{
  "subprocess_template_id": "manager-approval-v3",
  "inputs": [
    { "variable": "requester_id", "type": "uuid", "required": true },
    { "variable": "amount", "type": "number", "required": false },
    { "variable": "request_type", "type": "string", "required": true }
  ],
  "outputs": [
    { "variable": "decision", "type": "enum[approved,rejected]" },
    { "variable": "decision_note", "type": "string" },
    { "variable": "decided_by", "type": "uuid" }
  ]
}
```

چرخه‌ی کامل یک اجرا

[Trigger | Event | Schedule | فرم می‌دهد: کاربر | Trigger]

|



[Workflow Instance | Template ساخته می‌شود، روی آخرین نسخه‌ی منتشرشده‌ی Workflow Instance]

|



— End Node (تا رسیدن به) حلقه‌ی اجرای گراف —

| فعلی اجرا می‌شود Node |

| • Automation → بی‌درنگ |

| • Human (Task/Approval/Form) → Work Item ساخته می‌شود |

| • Decision/Condition → Rule Engine بدون Work Item صدا زده می‌شود، بدون |

| • Gateway → مسیر بعدی طبق شرط/موازی تعیین می‌شود |

| Exception یا مسیر Exception/SLA Breach → Escalation |

|

▼

[End Node → Instance = Completed | Terminated]

|

▼

[KPI به روزرسانی، Notification، Audit Log، رویدادهای پایانی]

Universal Process Core (هسته‌ی مشترک همه‌ی فرایندها)

بعد از بررسی همه‌ی ۲۰ فرایند کامل در data/process-library/sample-full- processes.json و همه‌ی الگوهای [۵۶](#)، این عناصر در تقریباً همه‌ی **Processها** (بدون استثنای معنادار) تکرار شدند:

عنصر	چرا Universal است
Request	هر Instance با یک درخواست/Trigger شروع می‌شود
Form	هر Request داده‌ی ساختاریافته جمع می‌کند
Task	واحد پایه‌ی کار انسانی، در همه‌ی ۲۰ نمونه حضور دارد
Assignment	هر Work Item به کسی باید برسد
Approval	در ۱۶ از ۲۰ نمونه‌ی کامل مستقیماً حاضر است
Decision	حتی وقتی پنهان است (مثل Threshold Routing)، در اکثر نمونه‌ها هست
Notification	هر نمونه حداقل یک Notify دارد
SLA	در ۱۴ از ۲۰ نمونه صریحاً تعریف شده
Escalation	زیرمجموعه‌ی طبیعی SLA، در نمونه‌های Human-in-the-loop تکرار می‌شود
Document	حداقل نصف نمونه‌ها یک سند تولید/مصرف می‌کنند

عنصر	چرا Universal است
Comment	در هیچ نمونه‌ای صریحاً مدل نشد اما در تمام Approval/Review Node ها به عنوان بخشی از <code>result</code> ضمنی حاضر است – باید صریح شود (نگاه کنید ۱۷ بخش MVP)
Attachment	فایل ضمیمه در فرم‌های Expense/Ticket/Audit تکرار شد
Audit Log	زیرساخت موجود پروژه (<code>audit_log</code>) از قبل این نقش را دارد – باید Workflow Engine هم به همان جدول بنویسد، نه جدول جدا
Status	هر Work Item و هر Instance یک وضعیت دارد
History	نتیجه‌ی طبیعی Audit Log + Version + Assignment History

نتیجه: این ۱۵ عنصر باید **هسته‌ی غیرقابل حذف Process Engine** باشند – یعنی هر Node/Pattern جدیدی که در آینده اضافه می‌شود باید از این هسته استفاده کند، نه اینکه دوباره Request/Task/Notification از صفر بسازد. این دقیقاً پاسخ به سؤال «چه چیزی نباید Hard-code شود» در [\(۱۰\)](#) Executive Summary است: خود این ۱۵ عنصر ثابت‌اند (Core)، اما محتوای هر کدام (کدام Form، کدام SLA، کدام Approver) کاملاً Configuration-driven است.

اسناد مرتبط: [۱۷-Product-۰۷](#) · [۱۵-Versioning](#) · [۰۹-Data Model](#) · [Node Catalog](#)
[Recommendations](#)

۰۹ – Data Model پیشنهادی

این سند خروجی بخش هشتم و نوزدهم درخواست کاربر است: طراحی مفهومی کامل موجودیت‌ها، نه SQL اجرashدنی (طبق قانون «هیچ Migration اجرا نشود»). Enum/روابط/Index دقیق برای هر Entity آمده تا تیم توسعه بتواند مستقیماً از روی این سند Migration بنویسد.

فهرست کامل Entityها (بازنگری‌شده نسبت به لیست اولیه‌ی کاربر)

لیست اولیه‌ی ۳۵ Entity کاربر پایه‌ی خوبی بود؛ بعد از Research چند اصلاح اعمال شد:

- `workflow_nodes / workflow_edges` به‌عنوان جدول جدا حذف شدند – طبق الگوی هر چهار موتور بررسی‌شده (Camunda/Zeebe/Step Functions/n8n)، گراف به‌صورت یک ساختار JSON منسجم داخل `workflow_versions.graph` نگهداری می‌شود، نه به‌صورت ردیف‌های نرمال‌شده در دو جدول – نرمال‌سازی گراف هزینه‌ی Join بالا برای سودی که در این مقیاس (چند صد Node در هر Template) وجود ندارد ایجاد می‌کند. `workflow_nodes` به‌عنوان **Catalog از انواع Node قابل‌استفاده** (نه نمونه‌ی واقعی هر Node) باقی می‌ماند – نگاه کنید به [۰۷](#).
- `workflow_templates / workflow_versions` طبق [۱۵-versioning](#) به‌صورت سه‌جدولی (Template/Version/Instance) اصلاح شد.
- `business_rules` اضافه شد (بخش کاربر آن را زیر Rule Engine در بخش ۱۴ خواسته بود ولی در لیست ۳۵ تایی نبود) – طبق الگوی DMN، یک Entity مستقل و نسخه‌بندی‌شده.
- `schedules / escalation_rules` اضافه شدند (طبق الگوی ServiceNow/Zendesk در [۱۲-sla](#) – تقویم کاری باید از خود SLA جدا باشد).
- `process_synonyms` اضافه شد (طبق الزام صریح کاربر در بخش ۲۶ برای مدیریت Synonym).

نمودار روابط سطح‌بالا

```
industries —|
                |—< processes >— process_types
domains ————|
```

```

functions —┐└< process_synonyms
              └< process_tags
              └1:0..N-< workflow_templates —1:N-< workflow_versions
—1:N-< workflow_instances
              |
              |
              |
              |└<
form_bindings └< work_items —< assignments
              |
              └<
rule_bindings └< process_variables
              |
└< escalations
form_templates —1:N— form_versions —< form_fields
business_rules —1:N— business_rule_versions —< business_rule_conditions
roles —< role_permissions (پروژه‌ی موجود)      departments (per-org)
slas —< sla_targets      schedules —< schedule_periods, schedule_holidays
documents, process_documents, process_kpis, process_metrics

```

گروه ۱ – Taxonomy (طبقه‌بندی، بدون گراف اجرا)

industries

ستون	نوع	توضیح
id	uuid PK	
slug	text UNIQUE	مثل <code>healthcare</code> , <code>saas</code>
name_en	text NOT NULL	
name_fa	text NOT NULL	
category	(enum(<code>cross_industry</code> , <code>industry_specific</code>	

ستون	نوع	توضیح
apqc_variant_available	boolean DEFAULT false	آیا APQC نسخه‌ی صنعتی مستقل دارد (نگاه کنید به ۳.۵)
parent_industry_id	uuid FK→industries NULL	برای زیرصنعت‌ها (مثلاً Property Casualty Insurance & Insurance) (Insurance

domains

ستون	نوع	توضیح
id	uuid PK	
slug	text UNIQUE	
name_en / name_fa	text NOT NULL	
apqc_category_code	text NULL	مثل ۹.۰ – ارجاع غیرالزامی به APQC Category
is_cross_functional	boolean	آیا این Domain معمولاً چند Department را قطع می‌کند

functions

ستون	نوع	توضیح
id	uuid PK	
domain_id	uuid FK→domains NOT NULL	
slug, name_en, name_fa		
apqc_process_group_code	text NULL	مثل ۹.۱
(INDEX (domain_id		

process_types

ستون	نوع	توضیح
id	uuid PK	

ستون	نوع	توضیح
slug	text UNIQUE	approval , incident , onboarding ... (نگاه کنید به ۵۰)
name_en, name_fa		
description	text	

processes (هسته‌ی Process Catalog – Taxonomy، نه اجرا)

ستون	نوع	توضیح
id	uuid PK	
slug	text UNIQUE	purchase-requisition
canonical_name_en	text NOT NULL	
canonical_name_fa	text NOT NULL	
description_en, description_fa	text	
domain_id	uuid FK→domains NOT NULL	
function_id	uuid FK→functions NULL	
process_type_id	uuid FK→process_types NOT NULL	
parent_process_id	uuid FK→processes NULL	برای Sub-process (خودارجاع)
apqc_process_code	text NULL	مثل 9.1.1
trigger_type	(enum(manual , scheduled , event , api	
complexity	(enum(simple , moderate , complex	
status	(enum(draft , active , deprecated	
version	integer DEFAULT 1	نسخه‌ی خودِ تعریف Taxonomy (نه Workflow – این دو مستقل‌اند)
source_confidence	(enum(high , medium , low	طبق قانون کاربر: بین Fact/Practice/Recommendation تفکیک قائل شو

ستون	نوع	توضیح
source_reference	text NULL	لینک/نام منبعی که این Process از آن استخراج شده
INDEX (domain_id), (process_type_id), (parent_process_id) (· UNIQUE(slug		

process_synonyms

id PK | process_id FK → processes | synonym_en text | synonym_fa text NULL | source text |
(NULL | UNIQUE(process_id , synonym_en

process_tags / تگ چندبه چند

process_tags(id, slug, name_en, name_fa) + process_tag_links(process_id
(process_id, tag_id) مرکب FK, tag_id FK) – PK

process_industries (چندبه چند: یک Process می تواند در چند صنعت کاربرد داشته باشد)

process_industries(process_id FK, industry_id FK, relevance
enum(core, common, rare)) – PK مرکب

گروه ۲ – Workflow Engine (اجرا)

نگاه کنید به [versioning-۱۵](#) برای
workflow_templates / workflow_versions / workflow_instances کامل. فیلد کلیدی
workflow_versions.graph (jsonb) ساختارش:

```
{
  "nodes": [{ "id": "n1", "type": "approval", "config": {...}, "position": {...} }],
  "edges": [{ "id": "e1", "from": "n1", "to": "n2", "condition": null } ]
}
```

work_items

ستون	نوع
id	uuid PK
instance_id	uuid FK→workflow_instances NOT NULL
node_id	text NOT NULL – ارجاع به id داخل graph.nodes ، نه FK پایگاه داده ای
type	(enum(task , approval , form , event_wait
status	ing , in_progress , completed , skipped , cancelled , escalated
assigned_to_user_id	uuid FK NULL
assigned_to_role_id	uuid FK NULL
assigned_to_queue	text NULL
due_at	timestampz NULL
completed_at	timestampz NULL
result	jsonb NULL – خروجی (تصمیم Approval، داده ی Form)
INDEX (instance_id), (assigned_to_user_id , status), SLA ((due_at حیاتی است	

process_variables

id PK, instance_id FK, key text, value jsonb, updated_at –
(UNIQUE(instance_id , key

assignments (work_item تاریخچه ی تخصیص/Reassign/Delegate – جدا از وضعیت فعلی روی

id PK, work_item_id FK, assigned_to_user_id FK, assigned_by_user_id FK,
reason enum(initial,reassign,delegate,escalate,round_robin), created_at

`form_templates` / `form_versions` / `form_fields`

همان الگوی پروژه‌ی فعلی (`form_templates` → `form_fields`) با یک تغییر: بین Template و Field یک لایه‌ی `form_versions` اضافه می‌شود تا طبق [versioning-۱۵](#) نسخه‌بندی مستقل داشته باشد. `form_fields.form_version_id` (نه مستقیم `form_template_id`).

Rules – ۴ (نگاه کنید به [rule-engine-۱۴](#) برای جزئیات)

`business_rules` / `business_rule_versions`

`business_rule_versions.hit_policy`

`rule_order, output_order, collect_sum, collect_min, collect_max, collect_count)`

– طبق DMN تأییدشده در Research

`business_rule_conditions` (ردیف‌های Decision Table)

<code>id,</code>	<code>rule_version_id</code>	<code>FK,</code>	<code>row_order</code>	<code>int,</code>	<code>input_conditions</code>	<code>jsonb,</code>
					<code>output_values</code>	<code>jsonb</code>

Roles / Assignment – ۵ (نگاه کنید به [۱۳](#))

`org_members`, `roles`, `departments` از پروژه‌ی موجود استفاده می‌شوند (`roles`, `org_members`, `departments`), `role_permissions` از قبل وجود دارند – این سند آن‌ها را بازتعریف نمی‌کند، فقط Work Item Assignment را به آن‌ها وصل می‌کند).

SLA – ۶ (نگاه کنید به [۱۲](#))

`slas`, `sla_targets`, `schedules`, `schedule_periods`, `schedule_holidays`,
`escalation_rules`, `escalation_levels`

Document / KPI / Metric – ۷ گروه

documents(id, slug, name_en, name_fa, category),
 process_documents(process_id FK, document_id FK, is_required boolean),
 process_kpis(id, process_id FK, name_en, name_fa, unit, target_value
 numeric NULL), process_metrics
 (Research, فقط جای رزرو شده)

استانداردهای Naming (پاسخ به بخش ۲۴)

مورد	قاعده‌ی انسانی (نمایش)	قاعده‌ی Machine-readable (شناسه)	مثال
Process	Title Case فعل+مفعول	kebab-case ، انگلیسی، فعل مصدری اول	نمایش: «Purchase Requisition» شناسه: purchase- requisition
Workflow Template	نام Process + توضیح تمایز اختیاری	{process-slug} یا {process- slug}- {variant}	purchase-requisition-- high-value
Workflow Version	—	عدد صحیح صعودی، همیشه کنار Template ID	purchase-requisition v3
Node (در گراف)	برچسب آزاد کاربر	{شمارنده n} یا UUID کوتاه، یکتا فقط در محدوده‌ی همان گراف	n7
Variable	camelCase در نمایش فنی، برچسب فارسی برای کاربر	snake_case ، بدون فاصله، پیشوند دامنه اختیاری برای تداخل‌زدایی	requester_manager_id
Form	نام Form Template	kebab-case	expense-claim-form
Rule/Decision	نام تصمیم، نه شرط	kebab-case ، پسوند نسخه در URL نه در نام	approval-threshold-by- amount

مورد	قاعده‌ی انسانی (نمایش)	قاعده‌ی Machine-readable (شناسه)	مثال
Event	فعل گذشته (چیزی که رخ داده)	PascalCase.PastTense (سبک (Domain Event	LeaveRequestApproved
Action (اقدام خودکار)	فعل امری	snake_case ، فعل امری	send_notification , create_task
Status	صفت/حالت کوتاه	snake_case ، از یک واژه‌نامه‌ی ثابت در سطح کل محصول (نگاه کنید به Universal Core در ۱۷)	pending_approval , in_progress
Role	اسم شغلی/ مسئولیت، نه شخص	kebab-case	department-head , process- owner

قاعده‌ی سراسری: انگلیسی همیشه Canonical Identifier است (طبق بخش ۲۷ کاربر)؛ فارسی فقط برای `name_fa / description_fa` ، هرگز در `slug / id` نام کد. این یعنی جابه‌جایی زبان نمایش هرگز شناسه‌ی پایگاه داده را نمی‌شکند.

اسناد مرتبط: [۷-Versioning-۱۵](#) · [۱۴-Rule Engine](#) · [۱۲-SLA](#) · [Node Catalog](#)

ساختار Library

Process Library

- └─ Human Capital (HR)
- └─ Financial Resources (Finance)
- └─ Supply Chain & Procurement
- └─ Information Technology
- └─ Marketing & Sales
- └─ Customer Service
- └─ Risk, Compliance & Resiliency (Legal/Compliance/Quality/Security)
- └─ Asset Management (Facilities/Maintenance)
- └─ Business Capabilities (Document/Knowledge Management, Project Management, Administration)
- └─ Vision & Strategy
- └─ Product & Service Development
- └─ Service Delivery
- └─ External Relationships
- └─ Industry-Specific (فیلترشده با Domain، زیرمجموعه‌ی اختیاری هر `process\_industries`)

این ساختار مستقیماً از ۱۳ Category تأییدشده‌ی APQC می‌آید (نگاه کنید [۰۴](#))، نه یک دسته‌بندی اختراعی – انتخابی که ریسک «Duplicate کردن یک استاندارد جاافتاده» را حذف می‌کند.

Metadata هر Process Template (طبق درخواست دقیق کاربر)

هر ردیف `processes.json` این فیلدها را دارد (نگاه کنید [data-model-۰۹](#) برای Schema کامل):
`id/slug`، `canonical_name_en/fa`، `description`، `domain_id`، `function_id`،

parent_process_id ، complexity ، status ، version ، (طبق ۵)، process_type_id
و process_synonyms به علاوه ، source_confidence ، source_reference
process_tags به صورت جدول‌های چندبه‌چند مرتبط. «Required»
processes ذخیره نشدند؛ چون این‌ها به ازای هر Workflow Template (پایاده‌سازی) می‌توانند فرق
کنند (نگاه کنید تفاوت Process/Workflow در ۲) – به‌جایش در سطح / workflow_templates
نمونه‌های کامل (Seed (sample-full-processes.json نگه داشته می‌شوند.

۲۰ نمونه‌ی کاملاً بازشده

طبق درخواست صریح کاربر، ۲۰ Process به‌طور کامل (Trigger/Actors/Steps/Forms/Rules/SLA) در
data/process-library/sample-full-processes.json آماده شدند. فهرست نهایی (با یک
جایگزینی آگاهانه نسبت به لیست پیشنهادی اولیه‌ی کاربر – نگاه کنید یادداشت پایین):

Leave Request · Purchase Requisition · Employee Onboarding · Employee Offboarding ·
Recruitment · Expense Reimbursement · Training Request · Incident Management · Change
Management · IT Access Request · IT Asset Request · Service Request Fulfillment · Vendor
Onboarding · Invoice Approval · Contract Review (Legal) · Audit Finding Remediation ·
Maintenance/Repair Request · Document Approval Cycle · Refund/Return Request ·
Customer Complaint Handling

یادداشت شفافیت: لیست پیشنهادی کاربر شامل «Project Change Request» بود. Research نشان داد
«Project Management» Domain در منابع Vendor واقعی (.../ServiceNow/Salesforce/SAP)
به‌اندازه‌ی کافی مستند نبود تا یک نمونه‌ی معتبر و غیرحدسی ساخته شود (نگاه کنید [gap-analysis-۱۶](#))
– به‌جایش «Employee Offboarding» (که منبع معتبر Vendor دارد) جایگزین شد تا طبق قانون صریح
کاربر («بین Fact و حدس تفاوت بگذار») چیزی حدسی وارد Seed Data نشود.

Reusability (پاسخ به بخش دوازدهم – «Manager Approval» به‌عنوان Subprocess در چند Process)

طبق [engine-architecture-۰۸](#)، الگوی فنی این کار Input/Output + is_callable_only=true
Contract است. در سطح Library، این یعنی:

workflow_templates

```
|— manager-approval (is_callable_only=true) ← یکبار تعریف
|— leave-request → Node "Subprocess" → manager-approval ارجاع به
|— purchase-requisition → Node "Subprocess" → manager-approval ارجاع به
|— expense-reimbursement → Node "Subprocess" → manager-approval ارجاع به
|— recruitment-requisition-to-hire → Node "Subprocess" → manager-approval ارجاع به
```

در `sample-full-processes.json` این چهار Process فعلاً Approval را Inline (نه به صورت Subprocess ارجاع شده) نشان می‌دهند – این یک تصمیم آگاهانه برای خوانایی Seed Data است؛ در پیاده‌سازی واقعی Builder، این چهار Approval باید به یک Template `manager-approval` مشترک تبدیل شوند (این خودش اولین Refactor پیشنهادی هنگام ساخت Builder واقعی است، نه یک اشتباه در Seed).

چگونه فایل‌های JSON به هم وصل می‌شوند

industries.json —

domains.json ————|< processes.json >— process-types.json

```
|
|
|   |< sample-full-processes.json (نمونه‌ی کامل، ۲۰ مورد)
|
|   |
|   |—< forms.json (ارجاع با slug)
|   |—< documents.json (ارجاع با slug)
|   |—< roles.json (ارجاع با slug)
|   |—< slas.json (ارجاع با slug)
|   |—< workflow-nodes.json (هر step.node_type)
```

workflow-patterns.json — workflow-nodes.json هرکدام ترکیبی از

process-steps.json — [#file-09-data-model](#) گاه كنيد ([۰۹] Node و Pattern لايه‌ی میانی بین

اسناد مرتبط: [۵-۱۶-Gap Analysis](#) · [۸-Engine Architecture](#) · [Process Taxonomy](#)

۱۱ – ارتباط Process Builder با سایر ماژول‌های محصول

Integration: Trigger، Input، Output، Data هر برای یک ماژول جزیره‌ای باشد. Contract، Dependency، Ownership

Calendar / Shift (ماژول موجود پروژه: calendar , shifts , holidays.ts)

یک Wait / Timer / Due Date Node در Workflow نیاز به محاسبه‌ی «چند روز کاری بعد» دارد	Trigger
تاریخ شروع + schedule_id (که باید معادل تقویم/شیفت همان Organization باشد)	Input
تاریخ/ساعت واقعی سررسید، با احتساب تعطیلات و ساعت کاری	Output
schedules / schedule_holidays (سند ۱۲) باید از همان منبع تعطیلات رسمی ایران که holidays.ts / calendar_events موجود پروژه دارند تغذیه شوند – نباید دو منبع حقیقت برای تعطیلات وجود داشته باشد	Data Contract
Process Engine بدون این ادغام هم کار می‌کند (Calendar Hours ساده) اما SLA دقیق (Business Hours) بدون آن ناقص است	Dependency
ماژول تقویم مالک داده‌ی تعطیلات/شیفت است؛ Process Engine فقط مصرف‌کننده است، نه Cache مستقل نگه می‌دارد	Ownership

Project Management (گام ۳ نقشه‌راه)

Node از نوع Create Record با نوع مقصد project / milestone / task	Trigger
عنوان، تاریخ‌ها، Owner، متغیرهای Instance که باید به فیلدهای پروژه Map شوند	Input
شناسه‌ی Project/Milestone/Task ساخته‌شده، برگردانده به process_variables برای استفاده در Nodeهای بعدی	Output
Mapping فیلد صریح (نه ضمنی) طبق همان الگوی Contract-based Input/Output در ۸	Data Contract

مستقل – Project Module می‌تواند بدون Process Engine هم کار کند؛ این یک ادغام یک‌طرفه است (Workflow → Project، نه برعکس، حداقل در فاز اول)	Dependency
ماژول Project مالک Schema پروژه/Milestone است	Ownership

Task Manager

هر Task (Human Work Node) می‌تواند به صورت اختیاری یک کارت Task Manager هم بسازد (برای نمایش در Kanban ماژول مربوطه)، نه فقط work_items داخلی	Trigger
عنوان، Assignee، Deadline از خودِ Work Item	Input
وضعیت تکمیل Task Manager باید Work Item مربوطه را هم به روزرسانی کند (دوطرفه، برخلاف اکثر Integration‌های این سند)	Output
work_items.external_ref jsonb (اشاره به رکورد Task Manager) + Webhook/Event برعکس برای Sync وضعیت	Data Contract
طبق Task Manager، ROADMAP.md موتور جدا ندارد و باید از همین Workflow Engine استفاده کند – این یعنی این Integration در واقع «همان سیستم»، نه دو سیستم جدا با پل ارتباطی	Dependency
مشترک – این تنها Integration در این سند است که واقعاً دوطرفه و بدون مالک یکتا است	Ownership

Ticketing (ماژول موجود پروژه: tickets)

Create Ticket Node ، یا برعکس: تغییر وضعیت یک Ticket موجود می‌تواند یک Workflow Instance را Trigger کند (مثلاً «Ticket بسته شد → شروع فرایند نظرسنجی رضایت»)	Trigger
فیلدهای Ticket (که خودشان از form_fields موجود پروژه می‌آیند – دقیقاً همان Contract با ماژول Form Builder، تکراری نسان)	Input
tracking_code موجود پروژه، شناسه‌ی Ticket	Output
tickets.id ↔ workflow_instances.related_record_type='ticket', related_record_id	Data Contract
Ticketing موجود در پروژه از قبل کار می‌کند بدون Workflow Engine؛ این Integration باید افزایشی (Additive) باشد – رفتار فعلی Ticket را نشکند (طبق قانون کاربر «کدنویسی اپلیکیشن تغییر نکند» در	Dependency

همین (Research)	
ماژول Ticketing مالک داده‌ی Ticket است	Ownership

Form Builder (ماژول موجود پروژه: form_templates , form_fields)

نگاه کنید به [data-model-۰۹](#) بخش Forms و [versioning-۱۵](#) برای مدل نسخه‌بندی کامل. خلاصه:

Trigger	Form Node در گراف Workflow
Data Contract	هر فیلد Form با یک variable name مشخص، مستقیم به process_variables نگاشت می‌شود؛ نوع فیلد Form باید با نوع اعلام‌شده در ورودی Node بعدی سازگار باشد (اعتبارسنجی زمان طراحی، نه فقط زمان اجرا)
Dependency	Workflow Engine به Form Builder موجود پروژه وابسته است، نه برعکس — Form Builder باید بدون تغییر بماند، فقط یک لایه‌ی نسخه‌بندی (form_versions) رویش اضافه می‌شود

Document Management

Trigger	Create Record با نوع مقصد document ، یا Sub-pattern «Document Sign-off» (۰۶ الگو ۳)
Output	سند تولیدشده + وضعیت امضا/تأیید
Dependency	این ماژول در ROADMAP.md فعلی تعریف‌نشده — طبق gap-analysis-۱۶ یک Gap واقعی است، نه صرفاً Integration نظری

Correspondence (نامه‌های اداری)

Trigger	الگوی Document Sign-off + Notification
Dependency	طبق ROADMAP.md، این پروفایل «عملیات» بعداً روی همین موتور ساخته می‌شود (گام ۴)، بدون تغییر هسته

Timesheet

Trigger	Work Item از نوع خاص می‌تواند Start/Stop زمان را ثبت کند
Data Contract	این دقیقاً معادل attendance_logs Runtime موجود پروژه است – باید بررسی شود آیا Timesheet یک ماژول جدا لازم دارد یا می‌تواند روی همان attendance موجود بنا شود (توصیه: بنا شود، نه بازسازی)

Notification

Trigger	هر Node که به Notification نیاز دارد (Send Email / Send SMS / In-app ، نگاه کنید ۵۷)
Data Contract	یک صف Notification مرکزی (خارج از Scope این Research، اما باید به‌عنوان یک Entity/Service مستقل طراحی شود که همه‌ی ماژول‌ها از جمله Process Engine از آن استفاده کنند – نه اینکه هر Node مستقیم ایمیل بفرستد)
Ownership	این سرویس باید مالک واحد داشته باشد؛ Process Engine فقط صف را پر می‌کند

اسناد مرتبط: [۰۸-Gap Analysis-۱۶](#) · [۱۲-SLA](#) · [Engine Architecture](#)

۱۲ – مدل SLA و Escalation

یافته‌ی کلیدی: سه معماری متفاوت و معتبر برای Escalation

Research سه پیاده‌سازی واقعی و متفاوت پیدا کرد – این یعنی «یک استاندارد واحد» وجود ندارد و باید آگاهانه انتخاب کرد:

رویکرد	پیاده‌سازی واقعی	مزیت	عیب
الف) Escalation داخل خود تعریف SLA	ServiceNow – Escalation SLA/Workflow متصل است	یکجا قابل مدیریت با SLA	کمتر قابل‌استفاده‌ی مجدد بین چند SLA
ب) Escalation به‌عنوان لایه‌ی Automation جدا، بر مبنای «ساعت تا/از نقض SLA»	Zendesk – Trigger Breach واکنش نمی‌دهد، فقط Automation زمان‌محور	قابل‌استفاده‌ی مجدد در چند SLA، مستقل تغییر می‌کند	یک لایه‌ی اضافه برای پیچیدگی
ج) Escalation به‌عنوان Timer+Event درون خود گراف Workflow	BPMN/Camunda – Timer Boundary Event غیرقطع‌کننده Escalation Throw/Catch +	کاملاً قابل‌مشاهده در دیاگرام فرایند، منطق نزدیک محل استفاده	برای هر Workflow جدا تعریف می‌شود، کمتر Reusable در سطح سازمان

تصمیم برای این محصول: ترکیب (ب) + (ج). `escalation_rules` به‌عنوان Entity مستقل و قابل‌استفاده‌ی مجدد در سطح Organization تعریف می‌شود (مثل رویکرد Zendesk)، اما هر `work_item` که SLA دارد، در گراف Workflow با یک `Timer / Escalation Node` (نگاه کنید [۵۰](#)) به همان Rule ارجاع می‌دهد (مثل رویکرد BPMN) – یعنی منطق Escalation یک‌بار تعریف و در چند Workflow ارجاع داده می‌شود، اما در دیاگرام هر Workflow قابل‌مشاهده است.

مدل داده (تأییدشده از ServiceNow + Zendesk، هر دو Schedule را از SLA جدا می‌کنند)

```
schedules
    Reusable تقویم کاری، مستقل و --
    id, org_id, name, timezone
```

```

schedule_periods          -- بازه‌ی ساعت کاری هر روز هفته
    id, schedule_id FK, weekday int(0-6), start_time, end_time

schedule_holidays        -- بدون تعطیلی نیمه‌روز - طبق) روزهای تعطیل تمام‌روز
    id, schedule_id FK, date, name
    -- این جدول باید با ماژول تقویم/تعطیلات موجود پروژه Integration: نکته‌ی مهم
    (holidays.ts, calendar_events) یکی شود، نه موازی

slas                      -- نه نمونه‌ی اجراشده، Template/تعریف
    id, org_id, name_en, name_fa
    applies_to enum(work_item_type, process_id, ...) -- به چه چیزی وصل می‌شود
    schedule_id FK → schedules
    start_condition, pause_condition, stop_condition -- عبارت شرطی (نه Hardcode)

sla_targets              -- (چند متریک: Zendesk: شبیه) SLA چند هدف روی یک
    id, sla_id FK, metric enum(first_response, resolution, ...), duration_minutes,
    priority_level

-- Runtime (نه Template): نمونه‌ی

work_item_slas           -- واقعی work_item به یک SLA اتصال
    id, work_item_id FK, sla_id FK, target_id FK
    started_at, paused_at, resumed_at, due_at, breached_at, stopped_at

escalation_rules         -- Reusable، خاص SLA مستقل از یک
    id, org_id, name_en, name_fa, applies_to

escalation_levels        --
    id, escalation_rule_id FK, level int, trigger_after_minutes int (نسبت به due_at)
    action enum(notify, reassign, reassign_and_notify, change_priority)
    target_role_id FK NULL, target_user_id FK NULL

```

چرا Pause/Resume یک شرط جدا از Stop است

هرکدام (`start_condition` / `pause_condition` / `stop_condition` / `reset_condition`) مستقل روی SLA Definition): یک درخواست می‌تواند «منتظر پاسخ مشتری» باشد بدون آنکه SLA لغو شود – ساعت متوقف می‌شود اما تاریخچه باقی می‌ماند. اگر Pause را با Stop قاطی کنیم، گزارش «چقدر واقعاً طول کشید» غیرقابل اعتماد می‌شود. Pause/Resume باید Timestamp جداگانه ثبت کند (`paused_at` , `resumed_at`) می‌تواند چند بار تکرار شود – در MVP یکبار کافی است، برای Enterprise باید آرایه/جدول جدا شود).

Calendar Hours در برابر Business Hours

هر `sla_targets.duration_minutes` باید مشخص کند نسبت به کدام محاسبه می‌شود:

- **Calendar hours** – زمان واقعی، بدون توجه به ساعت کاری/تعطیلات
- **Business hours** – فقط زمانی که داخل `schedule_periods` و خارج از `schedule_holidays` است

این تمایز مستقیماً از Zendesk گرفته شده (هر Target می‌تواند Business یا Calendar باشد، نه کل SLA یکجا).

اتصال به مازول تقویم/شیفت موجود پروژه (پاسخ صریح به بخش ۱۶ کاربر)

پروژه‌ی `org_platform` از قبل `calendar_events` ، `holidays.ts` ، و شیفت‌ها را دارد. `schedules` نباید تقویم تعطیلات ایران را دوباره بسازد – باید `schedule_holidays` را از همان منبع (Sync) `calendar_events` / `holidays.ts` یا مستقیماً Query کند. این دقیقاً یکی از موارد Cross-Module Integration که در [integration-model-۱۱](#) به‌طور کامل توضیح داده شده.

اسناد مرتبط: [۰۷-Integration Model](#) · [۰۹-Data Model](#) · [Node Catalog](#)

۱۳ – مدل Role و Assignment

یافته‌ی کلیدی: Assignment معمولاً خاصیت Work Item است، نه یک Entity جدا – به جز دو استثنای مهم

طبق Research (مقایسه‌ی Camunda 8، ServiceNow، Pega، jBPM): در بیشتر موتورهای Assignment یک Attribute روی خود Task است که هنگام فعال‌شدن Node محاسبه می‌شود (مثل zeebe:AssignmentDefinition روی User Task). دو استثنای مهم که باید Entity مستقل باشند:

1. **ServiceNow sysrule_assignment** – یک جدول قانون مستقل، جدا از Task، چون قوانین Assignment باید بدون لمس هر Workflow به صورت متمرکز قابل ویرایش باشند.
2. **Pega Operator Skill Ratings** – روی رکورد Operator زندگی می‌کند (نه روی Task)، چون یک مهارت باید در همه‌ی فرایندها یکسان باشد.

روش‌های Assignment (همه‌ی موارد خواسته‌شده کاربر، تأییدشده از منابع واقعی)

روش	تعریف	منبع تأیید
Specific User	Assignee = یک شناسه‌ی کاربر ثابت	Camunda 8 assignee (رشته یا FEEL Expression)
Role	هر عضو یک Role می‌تواند Claim کند (اولین نفر برنده)	Camunda 8 candidateGroups
Department	مثل Role، اما محدوده‌ی سازمانی به جای عملکردی	ترکیب Role + org_members.department
Manager / Supervisor	Assignee = مدیر مستقیم یک Actor مشخص (معمولاً Requester)	الگوی مدیریت سلسله‌مراتبی – Concur Management Chain Routing
Requester	Assignee = همان کسی که Instance را شروع کرده	استاندارد در همه‌ی الگوهای Rework/Resubmission
Requester's Manager	Expression پویا: مدیر مستقیم Requester	Camunda 8 FEEL Expression + پروژه‌ی موجود (org_members.manager_id)

روش	تعریف	منبع تأیید
Process Owner	نقشی که مالک تعریف یک Process Template است (نه لزوماً درگیر هر Instance)	Pega Case Type Owner
Previous Assignee	همان کسی که Node قبلی را انجام داده (برای زنجیره‌های پیوسته)	الگوی عمومی BPM
Dynamic User	نتیجه‌ی یک Rule/Expression در لحظه‌ی اجرا	Camunda 8 FEEL Expression روی assignee
Team	چند کاربر مشخص، همه Notify می‌شوند، یکی Claim می‌کند	Camunda candidateUsers
Queue	Work Item بدون Assignee اولیه در صف می‌ماند؛ اولین کارگر/کاربر آزاد Fetch-and-Lock می‌کند	Camunda 7 External Task Pattern (Fetch-and-Lock با Timestamp Lock ضدتداخل)
Round Robin	چرخشی بین اعضای یک گروه	ServiceNow «Round Robin Group» فیلد روی گروه
Load Balancing	بر اساس بار فعلی/در دسترس بودن Actor	ServiceNow Advanced Work Assignment (AWA) – «Last Assigned»/«Most Majority» + آگاهی از در دسترس بودن Agent
Skill-Based	بر اساس امتیاز مهارت (۱-۱۰) روی رکورد Actor	Pega ToSkilledGroup – انتخاب تصادفی از اعضای واجد حداقل امتیاز، Fallback به مدیر گروه اگر کسی Match نشود

Delegation (غیبت موقت – پاسخ صریح به نیاز کاربر)

دو الگوی متفاوت و هر دو معتبر:

- **الگوی Salesforce (Approval-محدود):** یک فیلد delegated_approver_id روی User؛ Delegate می‌تواند تصمیم بگیرد اما نمی‌تواند خودش دوباره Reassign کند – جلوگیری از زنجیره‌ی بی‌پایان Delegation.
- **الگوی jBPM (Deadline-محور):** اگر Task تا reassignment_time مشخص شروع/تمام نشود، خودکار به یک User/Group تعریف شده منتقل می‌شود.

تصمیم محصول: هر دو با هم پیاده‌سازی شوند: `delegations` (که از قبل در Schema پروژه با هدف دیگری وجود دارد – باید بازبینی/تطبیق شود، نه بازسازی) یک Actor را برای یک بازه‌ی زمانی به Delegate وصل می‌کند؛ موتور Assignment قبل از هر تخصیص جدید چک می‌کند آیا Assignee فعلاً Delegate فعال دارد یا نه، و طبق قانون Salesforce، خود Delegate اجازه‌ی Re-delegate ندارد.

مدل داده

```
assignment_rules          -- مستقل از هر ServiceNow، sysrule_assignment معادل
Workflow خاص
    id, org_id, name, applies_to_process_id FK NULL
    method enum(specific_user,role,department,manager,requester,requester_manager,
                process_owner,previous_assignee,dynamic_expression,team,queue,
                round_robin,load_balanced,skill_based)
    config jsonb          -- نیازمند skill (role_id، expression، skill
... )
    priority_order int    -- ServiceNow "order" کدام قانون اول ارزیابی شود، طبق الگوی
(lower runs first)"

skill_ratings             -- استفاده شود skill_based فقط اگر
    id, user_id FK, skill_slug, rating int(1-10)

-- به علاوه ،data-model از ۰۹ assigned_to_* work_items:
assignment_queue_membership
    id, queue_slug, user_id FK, is_available boolean    -- Round Robin/Load
Balancing
```

قاعده‌ی طلایی طراحی

Assignment Method باید همیشه یک **Fallback** صریح داشته باشد (طبق الگوی Pega: «اگر کسی Match نشد، به مدیر گروه برو»). یک Work Item بدون Assignee قابل حل و بدون Fallback، یک Silent

Failure است – دقیقاً نوع مشکلی که در Audit امنیتی پروژه‌ی فعلی (بخش «Silent Failures») هشدار داده شده بود.

اسناد مرتبط: [۷-Data Model-۰۹](#) · [۱۲-SLA](#) · [Node Catalog](#)

۱۴ – Rule Engine (جدا از Workflow)

اصل تفکیک (تأییدشده از استاندارد DMN + پیاده‌سازی واقعی)

طبق Research مستقیم DMN (استاندارد OMG که با BPMN جفت می‌شود) و پیاده‌سازی Camunda: یک Business Rule Task در گراف Workflow فقط یک ارجاع به یک Decision نگه می‌دارد (decisionId + resultVariable + bindingType)، نه خود منطق. این یعنی Rule Engine باید یک زیرسیستم کاملاً مستقل باشد که Workflow آن را «صدا می‌زند»، نه بخشی از گراف.

```
<!-- تأییدشده از مستندات رسمی، Camunda 8 نمونه‌ی واقعی -->
<bpmn:businessRuleTask id="determine-approver">
  <bpmn:extensionElements>
    <zeebe:calledDecision decisionId="approval_threshold"
resultVariable="approverRole" bindingType="latest" />
  </bpmn:extensionElements>
</bpmn:businessRuleTask>
```

ساختار Decision Table (تأییدشده از DMN)

هر Decision Table سه بخش دارد:

- **Input clauses** – ستون‌های شرط (چند ستون، ترکیب ضمنی AND بین ستون‌ها در هر ردیف)
- **Output clauses** – ستون‌های نتیجه
- **Rules** – ردیف‌ها، هر کدام یک نگاشت شرط → نتیجه

Hit Policy (تأییدشده، ۷ نوع طبق DMN 1.3)

کد	نام	معنی	کاربرد نمونه
U	Unique	فقط یک ردیف مجاز به Match است؛ همپوشانی خطای طراحی است	آستانه‌ی مبلغ با بازه‌های غیرهم‌پوشان

کد	نام	معنی	کاربرد نمونه
A	Any	چند ردیف می‌توانند Match کنند اما باید همه یک خروجی یکسان بدهند	قوانین افزونه‌ی هم‌جهت
P	Priority	چند Match مجاز؛ خروجی بر اساس اولویت از پیش تعریف‌شده در لیست مقادیر انتخاب می‌شود	وقتی چند قانون هم‌زمان صادق‌اند ولی یکی مهم‌تر است
F	First	اولین ردیف Match در ترتیب جدول برنده است	قوانین Fallback زنجیره‌ای
R	Rule Order	همه‌ی خروجی‌های Match شده، به ترتیب ردیف جدول برگردانده می‌شوند	جمع‌آوری چند اقدام هم‌زمان
O	Output Order	همه‌ی خروجی‌ها، به ترتیب اولویت خروجی برگردانده می‌شوند	مشابه R با اولویت‌بندی خروجی
C	Collect (+SUM/MIN/MAX/COUNT)	همه‌ی خروجی‌ها جمع‌آوری/تجمیع می‌شوند	محاسبه‌ی مجموع تخفیف از چند قانون

پیش‌فرض پیشنهادی محصول: Unique – چون بیشتر موارد کسب‌وکاری واقعی (آستانه‌ی مبلغ، تخصیص بر اساس دپارتمان) طبیعتاً بازه‌های غیرهم‌پوشان‌اند و Unique زودترین محل تشخیص خطای طراحی (همپوشانی ناخواسته) است.

نمونه‌ی عملی: دو مثال دقیقاً خواسته‌شده در Prompt کاربر

«اگر مبلغ < ۱۰۰,۰۰۰,۰۰۰ ریال، تأیید مدیر ارشد لازم است»

```
{
  "rule_slug": "approval-threshold-by-amount",
  "hit_policy": "unique",
  "inputs": [{ "variable": "amount", "type": "number" }],
  "outputs": [{ "variable": "required_approver_role", "type": "string" }],
  "rules": [
    { "conditions": { "amount": "> 100000000" }, "outputs": {
      "required_approver_role": "director" } },
  ]
}
```

```

    { "conditions": { "amount": "<= 1000000000" }, "outputs": {
"required_approver_role": "manager" } }
  ]
}

```

«اگر دپارتمان = IT، به مدیر IT تخصیص بده» (شکل ساده‌ی Condition Builder، طبق الگوی مشاهده‌شده در Salesforce Flow/ServiceNow Business Rules – فیلد/عملگر/مقدار)

```

{
  "rule_slug": "assignment-by-department",
  "hit_policy": "first",
  "inputs": [{ "variable": "department", "type": "string" }],
  "outputs": [{ "variable": "assign_to_role", "type": "string" }],
  "rules": [
    { "conditions": { "department": "= IT" }, "outputs": { "assign_to_role": "it-
manager" } },
    { "conditions": { "department": "= Finance" }, "outputs": { "assign_to_role":
"finance-manager" } }
  ]
}

```

Condition Builder (لایه‌ی UI برای کاربر غیرتوسعه‌دهنده)

طبق Research (Salesforce Flow Decision element؛ Medium Confidence روی نام دقیق فیلدها، اما شکل مفهومی با اطمینان بالا تأیید شد)، هر شرط یک ردیف با این شکل است:

```

{
  "conditionLogic": "and",
  "conditions": [
    { "field": "department", "operator": "equals", "value": "IT" },

```

```
{ "field": "amount", "operator": "greater_than", "value": 100000000 }  
]  
}
```

عملگرهای حداقلی لازم: `equals`, `not_equals`, `greater_than`, `less_than`, `greater_or_equal`, `less_or_equal`, `contains`, `is_empty`, `is_not_empty`, `in_list`. گروه‌بندی با `and / or / custom` (فرمول آزاد برای کاربر پیشرفته، دقیقاً الگوی (Salesforce).

چرا Drools/Salience اینجا استفاده نمی‌شود

Research موتور Drools را هم بررسی کرد (Salience برای اولویت، Agenda Group برای دسته‌بندی اجرای قوانین). این مدل برای سیستم‌های با **صدها قانون مستقل و متقاطع** مناسب است (مثل موتور تشخیص کلاهبرداری). برای این محصول – که هر Decision معمولاً به یک تصمیم مشخص محدود است (آستانه‌ی تأیید، تخصیص) – مدل **DMN Decision Table** با **Hit Policy** ساده‌تر، قابل‌نمایش بصری (جدول، نه قانون متنی)، و برای کاربر غیرفنی قابل فهم‌تر است. Drools/Salience را به‌عنوان الگوی مرجع برای فاز Enterprise (اگر روزی نیاز به قوانین متقاطع پیچیده شد) نگه می‌داریم، نه برای MVP.

اتصال به Data Model

`business_rule_versions` → `business_rules` (هرکدام `hit_policy` + آرایه‌ی `business_rule_conditions`) – نسخه‌بندی مستقل، دقیقاً مثل Workflow (نگاه کنید [۱۵-](#) [versioning](#)) تا تغییر یک آستانه نیازی به انتشار نسخه‌ی جدید کل Workflow نداشته باشد وقتی Binding از نوع `latest` است.

اسناد مرتبط: [۷-Versioning](#) · [۹-Data Model](#) · [Node Catalog](#) (Decision/Condition Node)

۱۵ – مدل Versioning (Template در برابر Instance)

یافته‌ی مرکزی Research

سه موتور واقعی (Camunda 7، Camunda 8/Zeebe، AWS Step Functions) و یک موتور Code-محور (Temporal) بررسی شدند. با وجود تفاوت در پیاده‌سازی، الگوی رفتاری هر چهار مورد یکسان است:

نسخه‌ی جدید = رکورد جدید و غیرقابل‌تغییر. Instance‌های در حال اجرا روی نسخه‌ای که با آن شروع شده‌اند «سنجاق» می‌مانند. انتقال به نسخه‌ی جدید هرگز خودکار نیست – همیشه یک عملیات صریح و جداگانه با نگاشت عنصر-به-عنصر است.

موتور	چگونه Version تعریف می‌شود	رفتار پیش‌فرض Instance موجود هنگام انتشار نسخه‌ی جدید	مکانیزم Migration
Camunda 7	Deploy مجدد همان process key → شماره‌ی نسخه خودکار +1	بدون تغییر؛ instance جدید همیشه روی آخرین نسخه شروع می‌شود	<code>RuntimeService</code> + Migration Plan (نگاشت) → Activity ID مبدأ (مقصد، فقط هم‌نوع)
Camunda 8 / Zeebe	<code>version</code> → Deploy عدد صحیح، خودکار توسط موتور تخصیص	بدون تغییر؛ <code>processDefinitionKey / version</code> instance ثابت می‌ماند	Process Instance Migration – Migration Plan با «Instructions» عنصر-به-عنصر؛ متغیرهای 全局 منتقل می‌شوند
AWS Step Functions	<code>UpdateStateMachine</code> تعریف را جای‌گزین می‌کند؛ با <code>publish=true</code> یک Version غیرقابل‌تغییر و شماره‌گذاری شده می‌سازد	Execution‌های در حال اجرا با نسخه‌ی قدیم تمام می‌شوند؛ Execution‌های جدید (چند ثانیه بعد) نسخه‌ی جدید می‌گیرند	بدون مفهوم Migration اصلاً – یک Execution همیشه با همان Snapshot تمام می‌شود؛ به‌جایش Alias (اشاره‌گر قابل‌جابه‌جایی بین حداکثر ۲ نسخه، برای

موتور	چگونه Version تعریف می‌شود	رفتار پیش‌فرض Instance موجود هنگام انتشار نسخه‌ی جدید	مکانیزم Migration
			(Canary/Blue-Green دارند)
Temporal	بدون جدول Definition/Version؛ نسخه‌بندی از طریق Worker Patching یا Versioning (شاخه‌ی <code>if(patched)</code> در <code>کد</code>)	Executionهای در حال اجرا باید دقیقاً همان مسیر <code>کد</code> را <code>Replay</code> کنند؛ <code>کد</code> جدید فقط شاخه‌های جدید می‌گیرد	نسخه‌بندی درون‌کدی، نه سطح داده — مرجع مقایسه‌ای، نه الگوی مستقیم قابل‌کپی برای این محصول

مدل داده‌ی پیشنهادی (سه جدول، نه دو)

workflow_templates

`id` (پایدار، هرگز عوض نمی‌شود)

`org_id`

`process_id` (FK → `processes`، Taxonomy پیوند به)

`slug`, `name_en`, `name_fa`

`is_callable_only` boolean

`current_published_version_id` (FK → `workflow_versions`، nullable تا اولین انتشار)

`status`: `draft` | `active` | `deprecated` | `archived`

`created_at`, `created_by`

workflow_versions

اجراشدنی؛ غیرقابل‌تغییر پس از انتشار `Definition` خود --

`id`

`template_id` (FK)

`version_number` (`template_id` صحیح، صعودی، منحصربه‌فرد در سطح)

`graph` jsonb -- `data-model` ها، نگاه کنید به `Node` ها/`Edge` ۰۹

`form_bindings` jsonb

`status`: `draft` | `published` | `deprecated`

`published_at`, `published_by`

change_note

workflow_instances

id

workflow_version_id (FK - است template_id نه ؛ این خود «سنجاق شدن»)

org_id

status: running | completed | terminated | migrated

variables jsonb

started_at, started_by, ended_at

چرا `workflow_instances.workflow_version_id` و نه `template_id` : اگر Instance به `template_id` وصل شود و گراف اجرا از «آخرین نسخه» خوانده شود، هر Instance در حال اجرا با هر انتشار جدید بی‌سروصدا رفتارش عوض می‌شود - دقیقاً نقطه‌ضعفی که هیچ‌کدام از چهار موتور بررسی‌شده اجازه‌اش نمی‌دهند. اتصال مستقیم به `workflow_versions.id` این خطا را از ریشه غیرممکن می‌کند.

جدول اختیاری Migration (فاز بعدی، نه MVP)

workflow_instance_migrations

id

instance_id (FK)

from_version_id (FK)

to_version_id (FK)

node_mapping jsonb -- [{ "from_node_id": "...", "to_node_id": "..."}]

migrated_at, migrated_by

طبق الگوی Camunda: نگاشت فقط بین Nodeهای **هم‌نوع** مجاز است (مثلاً Task→Task، نه Task→Gateway)؛ Instance‌هایی که در لحظه Migration داخل یک Node حذف‌شده در نسخه‌ی جدید هستند، باید صراحتاً رد یا به یک Node جایگزین نگاشت شوند - بدون نگاشت، Migration باید Reject شود، نه حدس زده شود.

Form Versioning (پاسخ به سؤال صریح کاربر در بخش ۱۳)

همان اصل: `form_templates` → `form_versions` (نه ویرایش In-place). یک `form_versions.id` به `workflow_versions.form_bindings` مشخص ارجاع می‌دهد، نه به `form_templates.id`. یعنی اگر Form بعداً تغییر کند (فیلد اضافه/حذف شود)، نسخه‌ی منتشرشده‌ی Workflow که از قبل به نسخه‌ی قدیم Form پین شده، دست‌نخورده می‌ماند؛ برای استفاده از Form جدید، باید نسخه‌ی جدید Workflow هم منتشر شود – همان قانون طلایی، بدون استثنا برای Form.

Rule/Decision Versioning

طبق الگوی DMN (سند ۱۴)، Decision Table ها هم خودشان نسخه‌بندی مستقل دارند (`business_rules` → نسخه‌های خودشان)؛ یک Business Rule Task در `workflow_versions.graph` می‌تواند به یک نسخه‌ی مشخص یا به «آخرین نسخه‌ی منتشرشده» ارجاع دهد (`bindingType: latest | version` – دقیقاً الگوی `zeebe:calledDecision` که در Research تأیید شد). Binding از نوع `latest` یعنی تغییر آستانه‌ی تأیید بدون نیاز به انتشار نسخه‌ی جدید Workflow اعمال می‌شود – این همان مزیت اصلی جداسازی Rule از Workflow است.

اسناد مرتبط: ۰۸-Rule Engine · ۱۴-Data Model · Engine Architecture

۱۶ – Process Completeness Analysis و بازبینی نهایی کیفیت

جدول Coverage (محاسبه شده از processes.json ، نه حدس)

شمارش واقعی از فایل Seed (Process ۵۱ کاتالوگ شده، اجرا شده با اسکریپت مستقیم روی JSON):

یادداشت	Coverage نسبت به ۱۳ Category	تعداد Process کاتالوگ شده	Domain (APQC Category)
پرمستندترین Domain در کل Research (Workday/BambooHR/SuccessFactors/Concur)	عمیق	10	Human Capital (7.0)
APQC ITSM (ServiceNow/ITIL) بسیار عمیق تر از خود Category 8.0 (نگاه کنید هه)	عمیق	8	Information Technology (8.0)
Tax/Treasury/Investor AP/AR/Close پوشش داده شد؛ Relations پوشش نشد	متوسط- عمیق	7	Financial Resources (9.0)
Procurement قوی؛ Manufacturing/Distribution فیزیکی پوشش نشد (چون این محصول برای خدمات/اداری طراحی شده، نه تولید)	متوسط- عمیق	7	Supply Chain & Procurement (4.0)
Campaign فقط CRM-محور (Salesforce/HubSpot)؛ Management/Brand عملاً پوشش نشد	کم-متوسط	4	Marketing & Sales (3.0)
یک مورد (Complaint Handling) با منبع ضعیف	کم-متوسط	4	Customer Service (6.0)
Legal/Compliance پوشش شد؛ Quality Management و Security به عنوان Domain مستقل پوشش نشد (فقط IT Security زیر Category 8.0 آمد)	کم	4	Risk, Compliance & Resiliency (11.0)
CMMS عمومی؛ Real Estate/Capital Asset پوشش نشد	کم	4	Asset Management (10.0)

یادداشت	Coverage نسبت به ۱۳ Category	تعداد Process کاتالوگ شده	Domain (APQC Category)
فقط Knowledge Management؛ Document Management و Project Management صفر Process مستند دارند	کم	3	Business Capabilities (13.0)
Research اصلاً به این Category نرسید – طبیعت این Category (تدوین استراتژی سطح بالا) کمتر شبیه یک Workflow اجرایی است، بیشتر یک فعالیت مدیریتی تک‌باره	صفر	0	Vision & Strategy (1.0)
همان دلیل – نیاز به Research مستقل در آینده اگر محصول به سمت PLM/NPD رفت	صفر	0	Product & Service Development (2.0)
همپوشان با IT/Customer Service شد، مستقلاً بررسی نشد	صفر	0	Service Delivery (5.0)
Research به این Category نرسید (روابط عمومی/سرمایه‌گذار – کم‌کاربرد برای MVP این محصول)	صفر	0	External Relationships (12.0)

نتیجه‌ی صادقانه (طبق قانون کاربر «Coverage را با منطق محاسبه کن، نه حدس»): از ۱۳ ۴ Category، **مورد صفر Process** دارند. این عمده‌اً پر نشد چون منابع Vendor معتبر برای آن‌ها در بازه‌ی این Research پیدا نشد – طبق قانون صریح کاربر («اگر منبعی نیست، حدس نزن») بهتر است این خلأ صادقانه گزارش شود تا اینکه با Process‌های ساختگی پر شود.

توزیع اطمینان منبع (Source Confidence)

از ۵۱ **۲۰ High، ۲۱ Medium، ۱۰ Low** Process. یعنی ~۲۰٪ کاتالوگ روی منابع کم‌عمق (عمده‌تاً الگوی عمومی «Generic Request Fulfillment» به‌جای مستندسازی مستقیم Vendor) بنا شده – این‌ها در `processes.json` با `source_confidence: "low"` صریحاً علامت‌گذاری شدند تا تیم توسعه بدانند کدام Process‌ها نیاز به بازبینی/Validation بیشتر قبل از اتکای کامل دارند: Goods Receipt، Payment Request، Facility Access Request، Asset Inspection، Customer Complaint

Handling, NDA Request, IT Asset Request, Document Publishing/Archival, Contract Approval (Sales), Overtime Approval

مقایسه‌ی Fact در برابر Industry Practice در برابر Recommendation (طبق قانون ۱۸ کاربر)

نوع	نمونه	چگونه مشخص شده
Fact	«APQC دقیقاً ۱۳ Category سطح یک دارد»	استخراج مستقیم از PDF رسمی APQC، اطمینان High
Industry Practice	«الگوی Approval معمولاً از Submit→Review→Approve→Execute→Close است»	مشاهده‌شده در چند Vendor مستقل (Concur/NetSuite/Salesforce)، اطمینان High-Medium
Recommendation	«Node Approval باید از Task جدا باشد» یا «Hit Policy پیش‌فرض Unique باشد»	قضاوت طراحی این Research بر مبنای شواهد، نه یک Fact مستقیم – در متن هرجا این نوع جمله آمد با عبارت «تصمیم پیشنهادی» علامت‌گذاری شد

بازبینی نهایی کیفیت (طبق درخواست صریح انتهای Prompt کاربر)

معیار	وضعیت	یافته
Completeness	⚠ جزئی	۴ APQC Category صفر Process دارند (بالا)؛ عمداً به‌جای پرکردن ساختگی خالی ماند
Consistency	✅	همه‌ی Process‌ها همان Schema را دارند (processes.json)؛ همه‌ی Node‌ها از یک واژگان ثابت (۷۵) استفاده می‌کنند
Duplicates	✅ رفع شد	«Purchase Requisition/Request/Procurement Request» و «Expense Reimbursement/Claim» Merge process_synonyms ثبت شدند (نگاه کنید ۵۵)
Missing Domains	⚠ مستندشده	بالا – ۴ Category خالی
Missing Process Types	✅ بررسی شد	همه‌ی ۲۴ Process Type درخواستی کاربر حداقل یک نمونه‌ی مرتبط در Catalog دارند، به‌جز «Registration» که فقط به‌صورت ضمنی (بخشی از Vendor Onboarding) حاضر است، نه یک Process مستقل – قابل‌قبول

معیار	وضعیت	یافته
		چون Registration معمولاً واقعاً زیرمجموعه‌ی یک Process بزرگ‌تر است، نه مستقل
Data Model Integrity	✓ بررسی شد	هر FK در ۹-۵ به یک جدول تعریف شده اشاره می‌کند؛ هیچ Orphan Reference پیدا نشد؛ همه‌ی JSON با <code>python3 -m json.tool</code> معتبرسنجی شدند
Scalability	✓ طراحی شده	جدول <code>graph jsonb</code> به‌جای نرمال‌سازی Node/Edge دقیقاً برای مقیاس معقول (چند صد Node در هر Template) انتخاب شد؛ <code>work_items</code> ایندکس روی <code>due_at</code> برای Scan مقیاس‌پذیر SLA دارد
Reusability	✓ طراحی شده	Input/Output Contract + <code>is_callable_only</code> (نگاه کنید ۸-۱۰)
Versioning	✓ طراحی شده	سه‌جدولی Template/Version/Instance، مبتنی بر رفتار واقعی چهار موتور مقایسه‌شده (نگاه کنید ۱۵)
Multi-tenancy	⚠ نیازمند هماهنگی صریح با پروژه‌ی موجود	همه‌ی جداول پیشنهادی <code>org_id</code> دارند، هماهنگی با الگوی RLS موجود پروژه؛ اما این Research مستقیماً RLS Policy پیشنهاد نداد چون خارج از Scope «فقط Research» بود – این باید اولین کار پیاده‌سازی باشد
Internationalization	✓ طراحی شده	<code>name_en / name_fa</code> روی هر Entity، انگلیسی Canonical (نگاه کنید ۹-۵ بخش Naming)
Extensibility	✓ طراحی شده	Node Catalog/Process Type/Domain همه Data-driven هستند (جدول، نه Enum سخت در کد)، یعنی افزودن Domain/Node جدید بدون تغییر Schema ممکن است

اصلاحات اعمال‌شده در همین بازبینی

در حین بازبینی نهایی، این موارد اصلاح شد (نه صرفاً گزارش شد، طبق دستور صریح «مشکلات کشف‌شده را اصلاح کن»):

- لیست ۲۰ نمونه‌ی کامل کاربر شامل «Project Change Request» بود که منبع معتبر نداشت – با «Employee Offboarding» (منبع High-confidence) جایگزین شد؛ این جایگزینی و دلیلش در [۱۰-۱](#) [process-library](#) صریحاً مستند شد تا شفاف بماند، نه پنهان.
- «Universal Process Core» (بخش بیست‌ودوم Prompt) در ابتدا در هیچ فایل‌ی جا نگرفته بود – به [engine-architecture-۰۸](#) اضافه شد.

3. بررسی شد که آیا `workflow_nodes / workflow_edges` باید جدول‌های نرمال‌شده باشند (طبق لیست اولیه‌ی کاربر) یا JSON – بر اساس شواهد چهار موتور واقعی (هیچ‌کدام Node/Edge را در جدول‌های SQL جدا نرمال نمی‌کنند) به `graph jsonb` تغییر یافت؛ این تغییر و دلیلش صریحاً در [data-model-۰۹](#) مستند شد.

اسناد مرتبط: [۰۵-Product Recommendations-۱۷](#) · [۱۰-Process Library](#) · [Process Taxonomy](#)

۱۷ – توصیه‌های محصول: MVP در برابر V2 در برابر Enterprise، و مقایسه با پروژه‌ی فعلی

بخش الف – MVP / V2 / Enterprise

Business Value	Complexity	Priority	سطح	Capability
بسیار بالا	متوسط	P0	MVP	Workflow Template + Version (بدون Migration)
بسیار بالا	متوسط	P0	MVP	Node: Start/End/Task/Approval/Decision/Branch/Parallel/Merge
بسیار بالا	کم (زیرساخت از قبل هست)	P0	MVP	Form Node + اتصال به Form Builder موجود
بسیار بالا	کم	P0	MVP	Assignment: Specific User/Role/Requester/Requester's Manager
بسیار بالا	کم	P0	MVP	Work Item + وضعیت پایه (pending/completed)
بالا (امنیت)	کم	P0	MVP	Audit Log Integration (نوشتن به audit_log موجود)
بالا	متوسط	P1	MVP	Rule Engine ساده (Decision Table) تک‌ورودی، Hit (Policy=Unique)
بالا	متوسط	P1	V2	SLA پایه (یک Target، بدون Pause/Business Calendar)
بالا	متوسط	P1	V2	Escalation چندسطحی
متوسط	بالا	P2	V2	Node: Timer/Wait/Loop/Multi-Instance
بالا (کاهش تکرار الگوی Approval)	بالا	P1	V2	Reusable Subprocess (is_callable_only)
متوسط	بالا	P2	V2	Assignment: Round Robin/Load Balancing/Skill-Based

Business Value	Complexity	Priority	سطح	Capability
متوسط	بالا	P2	V2	Business Calendar کامل (Business) (Hours/Holidays/Pause-Resume)
متوسط	متوسط	P2	V2	Delegation (غیبت موقت)
کم (فقط سازمان‌ها; بزرگ با Instance طولانی‌مدت لازم دارند)	بسیار بالا	P3	Enterprise	Process Instance Migration
بالا (برای مشتریان بزرگ)	بالا	P2	Enterprise	Webhook/API Call Node + Integration بیرونی
متوسط	بسیار بالا (ریسک امنیتی Sandbox)	P3	Enterprise	Script/Function Node (Sandbox)
بسیار بالا	بسیار بالا	P1	MVP-تا-V2 (مرحله‌ای)	Round-trip Editor بصری (Drag & Drop Builder)
متوسط	متوسط	P2	V2	Process Library UI (مرور/جستجو/فیلتر Catalog)
متوسط	بالا	P3	Enterprise	Analytics/KPI Dashboard روی Process Metrics

بخش ب – Universal Process Builder Capability Matrix

چرا	Enterprise	V2	MVP	Capability
بدون آن هیچ Process ای داده جمع نمی‌کند			✓	Form
پرتکرارترین Node در ۲۰ نمونه‌ی کامل (۱۰)			✓	Approval (تک‌سطحی)

چرا	Enterprise	V2	MVP	Capability
نیاز به Assignment پیچیده تر و UI بیشتر دارد		✓		Approval (چندسطحی/موازی)
لازم برای حتی ساده ترین Branching			✓	Decision (شرط ساده)
تا وقتی شرطها ساده اند، نیاز فوری نیست		✓		Decision) Rule Engine (Hit Policy با کامل Table
اکثر فرایندهای ساده خطی اند؛ فقط - Onboarding type نیاز دارند		✓		Parallel/Merge
ارزش بالا اما بدون Business Calendar کامل، دقتش محدود است		✓		SLA (پایه)
وابسته به SLA		✓		Escalation
ارزش بالا اما بدون Template پایدار خطرناک است (نگاه کنید ۱۵-versioning)		✓		Reusable Subprocess
نیاز واقعی اما نه برای اولین انتشار		✓		Delegation
فقط سازمان های با Instance های بسیار طولانی نیاز دارند	✓			Version Migration
ریسک امنیتی بالا، نیاز به زیرساخت Secret Management	✓			API/Webhook Integration
بالاترین ریسک امنیتی کل کاتالوگ Node	✓			Script/Function (Sandbox)
داده از ابتدا name_en / name_fa دارد، هزینه ی اضافه ی UI کم است			✓ (فقط fa/en)	Multi-language Process Library UI
وابسته به هماهنگی با ماژول تقویم موجود که هنوز کامل نیست (نگاه کنید Gap در بخش پ)		✓		Business Calendar کامل

بخش پ – Current Project vs Target Architecture (پاسخ صریح به دستور کاربر)

این بخش وضعیت واقعی `org_platform` (بر اساس بررسی مستقیم Schema/کد در همین Session) را با معماری پیشنهادی مقایسه می کند.

چه چیزی از قبل وجود دارد و مستقیماً قابل استفاده است

موجود در پروژه	چگونه به معماری پیشنهادی وصل می‌شود
organizations , org_members , roles , role_permissions , تابع has_permission / permission_scope	مستقیماً به‌عنوان لایه‌ی Authorization زیر Assignment/Rule Engine استفاده می‌شود – نیازی به بازسازی نیست
form_templates , form_fields (با ۷ نوع فیلد طبق حافظه‌ی پروژه)	پایه‌ی form_templates / form_versions پیشنهادی؛ فقط یک لایه‌ی نسخه‌بندی روی آن اضافه می‌شود (۱۵)
audit_log (Append-only, Privilege-Revoked) طبق Audit امنیتی قبلی همین Session)	Workflow Engine باید مستقیماً به همین جدول بنویسد، نه جدول موازی
tickets , ticket_field_values , tracking_codes	Ticket فعلی = یک نوع خاص و ساده از Case (نگاه کنید ۵۲)؛ می‌تواند اولین ماژول باشد که Workflow روی آن سوار می‌شود بدون بازنویسی
org_members.manager_id (سلسله‌مراتب مدیریتی، اضافه‌شده در گام ۲)	مستقیماً پایه‌ی «Requester's Manager» Assignment (۱۳) است – از قبل آماده است
calendar_events , holidays.ts , شیفت‌ها	پایه‌ی schedules / schedule_holidays پیشنهادی (۱۲) – باید Sync شود، نه بازسازی

چه چیزی Missing است (باید از صفر ساخته شود)

- workflow_templates / workflow_versions / workflow_instances / work_items
و کل موتور اجرا – هیچ‌کدام در پروژه‌ی فعلی وجود ندارد
- business_rules / Decision Table Engine – وجود ندارد
- assignment_rules متمرکز – فعلاً Assignment در پروژه (مثلاً تخصیص Leave Approval)
مستقیماً در کد Server Action Hardcode است، نه Data-driven
- slas / escalation_rules – وجود ندارد (Attendance/Leave فعلی SLA ندارند)
- Notification Queue متمرکز – فعلاً پروژه اصلاً هیچ Notification (نه ایمیل نه Toast) پیاده‌سازی
نکرده (طبق Audit امنیتی قبلی همین Session، Toaster نصب است اما هیچ‌جا صدا زده نمی‌شود)

چه چیزی باید Refactor شود

- **audit_log Trigger Coverage** : طبق Audit امنیتی قبلی همین Session، فقط tickets / leave_requests روی Trigger لاگ دارند. قبل از اتصال Workflow Engine به این جدول، باید این پوشش تکمیل شود – وگرنه اجرای Workflow روی جداول دیگر (مثل org_members) باز هم نامرئی می‌ماند.
- **Server Actions موجود** (shifts/actions.ts , leave/actions.ts , ...): این‌ها منطق Approval را Inline دارند. وقتی Workflow Engine ساخته شود، این منطق باید به تدریج به یک Workflow Instance واقعی منتقل شود، نه اینکه دو سیستم موازی (Hardcoded Approval قدیمی + Workflow Engine جدید) هم‌زمان زنده بمانند.

چه چیزی باید حذف شود

هیچ‌چیز – طبق قانون صریح کاربر («در این مرحله کدی تغییر نکنند»)، این Research هیچ کد موجودی را برای حذف پیشنهاد نمی‌دهد؛ فقط مسیر Migration تدریجی (بالا) را مشخص می‌کند.

چه چیزی فعلاً نباید ساخته شود

- Script/Function Node (ریسک امنیتی Sandbox بدون زیرساخت آماده)
- Webhook/Integration بیرونی (بدون Secret Management که در Audit امنیتی قبلی همین Session به عنوان Gap شناسایی شد)
- Process Instance Migration (پیچیدگی بسیار بالا برای ارزش کم در فاز اول)

چه بخش‌هایی برای MVP مناسب‌اند

جدول بخش الف بالا، ردیف‌های MVP – همگی روی زیرساخت موجود پروژه (Org/Role/Form/Audit) Log سوار می‌شوند، بدون نیاز به زیرساخت جدید پرریسک.

چه بخش‌هایی معماری آینده را محدود می‌کنند (اگر اشتباه ساخته شوند)

- اگر workflow_instances به جای workflow_version_id به workflow_template_id وصل شود (اشتباه رایج) – طبق [versioning-۱۵](#) این تصمیم برگشت‌ناپذیر است بدون Migration دردناک داده‌های تولیدشده.
- اگر Assignment مستقیم در کد Node به جای assignment_rules جدول مستقل نوشته شود – دقیقاً همان اشتباهی که Server Actions فعلی پروژه دارند و باید اصلاح شود، نه تکرار.

